

ProTech

Professional Services Marketplace Platform

Technical Report

End-of-Year Master's Project

IT Innovation in Business — Master's Degree

Web Technology in Business · Web Application Development

Author

Manye Patrice

Programme

IT Innovation in Business — Master's Degree

Project

End-of-Year Master's Project (Individual)

Date

June 09, 2026

Repository

github.com/logandescapo-eng/protech1

Table of Contents

Executive Summary

1. Introduction

- 1.1 Project Goals
- 1.2 Business Use Case
- 1.3 Problem Statement
- 1.4 Business Model and Monetisation Strategy
 - 1.4.1 Transaction Commission Model
 - 1.4.2 Worker Subscription Model
 - 1.4.3 Value Exchange and Break-Even Analysis
 - 1.4.3.1 Worker Subscription Break-Even
 - 1.4.3.2 Platform Operating Break-Even
 - 1.4.4 Alignment with the ProTech Implementation
 - 1.4.5 Go-to-Market and Growth Considerations
 - 1.4.6 Growth Projections: Traffic and Economics

2. System Design

- 2.1 Functional Architecture
- 2.2 Architecture Diagram
- 2.3 Use Cases
- 2.4 Key Features and Data Model
- 2.5 Entity-Relationship Model
- 2.6 Escrow Sequence Diagram

3. Implementation

- 3.1 Technology Choices
- 3.2 Application Structure
- 3.3 Security, Auth, and Escrow
- 3.4 Caching and Logging
- 3.5 Application Interface (Screenshot Placements)

4. Testing and Deployment

5. Conclusion, Limitations, and Literature

- 5.1 Conclusion
- 5.2 Limitations
- 5.3 Literature Review
- 5.4 References

Appendix A — Project Structure

Appendix B — Demo Credentials

Appendix C — Application Screenshots (Insert Images)

Appendix D — Rubric Alignment (200 Points)

Executive Summary

ProTech is a full-stack professional services marketplace developed by Manye Patrice as an end-of-year master's project within the IT Innovation in Business programme. The platform connects clients with skilled workers — plumbers, electricians, cleaners, and similar trades — through a single web application that handles discovery, booking, messaging, escrow-protected payments, reviews, and administrative oversight.

Technically, the system is built with Django 5.2.15, PostgreSQL 15, Redis 7, and Nginx, deployed as a four-service Docker Compose stack. It implements role-based access for clients, workers, and staff; a working escrow ledger with configurable platform commission; seventeen automated tests; structured logging; and a GitHub Actions CI pipeline. Evaluators can clone the repository and run `docker compose up --build` to access a fully interactive demo at <http://localhost>.

From a business perspective, ProTech proposes a hybrid monetisation model combining transaction commissions and tiered worker subscriptions, with explicit break-even analysis for both worker plans and platform operating costs, supported by illustrative twenty-four-month growth projections. The commission component is implemented in code; subscription tiers are specified for future production rollout. This report documents system design, implementation, testing, deployment, limitations, and the academic literature that informed key decisions.

1. Introduction

1.1 Project Goals

ProTech is a full-stack web application built as my end-of-year master's project for the courses Web Technology in Business and Web Application Development, part of the IT Innovation in Business programme. The aim is a secure, scalable marketplace that solves a real problem: helping households and small businesses find skilled workers such as plumbers, electricians, cleaners, and carpenters.

The project must show end-to-end skill in server-rendered front-end design, Django back-end development, database modelling, containerised deployment, automated testing, logging, and Redis caching. It also needs to be easy to run: anyone should be able to clone the GitHub repo, follow the README, start Docker Compose, and use a working marketplace with client and worker dashboards, bookings, escrow payments, and an admin console.

1.2 Business Use Case

In many urban and regional markets, the process of hiring a tradesperson remains informal and inefficient. Clients typically depend on personal referrals, unstructured social media posts, or costly agency intermediaries. They have limited ability to compare providers by skill, location, availability, or verified reputation before committing to a job. Payment is often negotiated in cash or via direct transfer with no dispute mechanism if work quality falls short.

Independent workers face the mirror image of this problem. Building a credible professional profile online requires technical skills many tradespeople do not possess. Managing inbound enquiries, scheduling, and payment collection across phone calls and messaging apps consumes time that could otherwise be spent on billable work. Reputation is hard to accumulate when satisfied customers have no structured channel to leave public reviews.

ProTech intervenes as a two-sided marketplace. Clients gain a searchable directory of workers, transparent rating data, integrated booking with date and location details, and escrow-protected payments that reduce upfront risk. Workers gain a hosted profile, inbound job requests, status tracking tools, wallet-based payouts on completion, and review-driven reputation growth. Platform operators use Django Admin to oversee users, service categories, bookings, and financial ledger activity, providing the governance layer expected of a commercial marketplace.

1.3 Problem Statement

Four interrelated problems motivate the design of ProTech. The discovery problem arises because clients cannot efficiently filter professionals by trade, geography, and verified ratings in a single interface. The trust problem follows from payment risk: without escrow, neither party is confident transacting online. The coordination problem reflects the fragmentation of booking status, messaging, and notifications across disconnected channels. Finally, the administrative problem stems from the absence of a central system to manage service taxonomy, user roles, and operational oversight at scale.

The solution brings discovery, scheduling, messaging, escrow payments, reviews, and admin tools into one Django application behind Nginx in Docker. That single integrated platform is the main contribution of the project, both as a business idea and as a technical build.

1.4 Business Model and Monetisation Strategy

A marketplace needs revenue without hurting either clients or workers. ProTech uses two income streams: a commission on each completed job, and monthly subscription plans for workers who want better visibility. Clients pay no membership fee — only the agreed job price, held in escrow until the work is done. Workers pay subscriptions if they want more leads, and commissions when jobs complete. That keeps sign-up easy for clients while workers who benefit most from the platform contribute more.

Commission alone scales with activity but can frustrate busy professionals who pay a lot yet get no search boost. Subscriptions alone give steady income but do not track actual job value. Platforms like Upwork, Thumbtack, and Fiverr mix both. ProTech does the same, and the commission part is already working in code: `escrow/services.py` applies `ESCROW_PLATFORM_FEE_PERCENT` when funds are released.

1.4.1 Transaction Commission Model

The primary revenue engine is a platform commission deducted automatically when escrow funds are released to the worker upon job completion. In the current implementation, the default rate is five percent of the gross booking price, configurable through the `ESCROW_PLATFORM_FEE_PERCENT` environment variable. When a worker marks a booking as complete and `release_escrow()` executes, the system calculates the platform fee, credits the worker's wallet with the net payout (gross amount minus fee), increments the `EscrowVault` released total, and records a `WalletTransaction` row for audit purposes. Clients are never charged an additional surcharge at checkout; the commission is borne by the worker as a cost of using the platform's payment infrastructure, dispute mediation capability, and demand aggregation.

To remain competitive across job sizes while preserving margin on small tasks, the production business model proposes a modestly tiered commission schedule rather than a single flat rate for all transactions. Standard residential jobs between fifty and five hundred currency units retain the five percent rate that is already implemented in the prototype. Micro-jobs below fifty units incur an eight percent rate or a minimum fee equivalent to three currency units, whichever is greater, because payment processing and support costs represent a larger share of economic value on small tickets. Commercial or repeat contracts above five hundred units qualify for a reduced three-and-a-half percent rate, rewarding high-value relationships that stabilise platform liquidity and reduce per-job acquisition cost.

Job value band	Commission rate	Example job	Platform fee	Worker net payout
Micro (under \$50)	8% (min. \$3)	\$40 plumbing call-out	\$3.20	\$36.80
Standard (\$50 – \$500)	5%	\$200 electrical repair	\$10.00	\$190.00

Commercial (over \$500)	3.5%	\$800 renovation phase	\$28.00	\$772.00
----------------------------	------	---------------------------	---------	----------

The commission is justified by tangible services rendered at the moment of payment release. ProTech holds funds in escrow, which reduces client risk of paying upfront for incomplete work and reduces worker risk of non-payment after labour is delivered. The platform maintains verified identity records, booking history, and review authenticity constraints that would be costly for an individual tradesperson to replicate. Customer support for cancellation disputes, chargeback documentation, and account recovery further justify a take-rate comparable to card-present merchant fees plus a modest marketplace premium.

1.4.2 Worker Subscription Model

The second revenue stream consists of recurring subscription plans targeted exclusively at workers. Clients remain free to register, search, book, and review because demand aggregation is the foundation of network effects; restricting client access would shrink the addressable audience that workers pay to reach. Workers who depend on ProTech for a meaningful share of their income require predictable visibility in search rankings, sufficient booking capacity, and professional signalling that differentiates them from casual or inactive profiles. Subscription tiers package these capabilities into clearly priced plans.

The Starter plan is free and exists primarily to onboard new professionals and populate the marketplace with searchable supply. Starter accounts receive a basic profile page and may accept up to five concurrent active bookings per calendar month. Their listings appear in search results but are ranked below all subscribed competitors, and profile impressions are soft-capped at approximately one hundred client views per month to prevent free accounts from consuming unlimited marketing exposure without contributing recurring revenue. Starter workers still pay transaction commissions on every completed job, ensuring the platform captures value when free-tier users convert demand into paid work.

The Professional plan, priced at twenty-nine dollars per month or two hundred ninety-five dollars annually after a fifteen percent annual discount, targets established tradespeople who require reliable lead flow. Subscribers receive standard search ranking priority above all Starter accounts, unlimited concurrent bookings, access to in-app messaging analytics showing response times and conversion rates, a verified Professional badge displayed on their profile, and a one percentage point reduction in transaction commission (four percent instead of five on standard-band jobs). This commission discount creates a financial incentive for active workers to subscribe once monthly job volume exceeds the break-even point described below.

The Premium plan, priced at fifty-nine dollars per month or five hundred ninety-nine dollars annually, is designed for workers who wish to maximise reach and dominate category visibility. Premium subscribers appear in a featured carousel at the top of browse_workers results within their declared service area, receive push-priority notification when new client searches match their skills, carry a Premium Verified badge, gain access to early promotional placement on the landing page success stories section, and benefit from a two percentage point commission reduction on standard-band jobs

(three percent instead of five). Unlimited client impressions and priority support response within twenty-four hours position Premium as the account type required for workers whose commercial strategy depends on ProTech as a primary channel rather than a supplementary referral source.

Plan	Monthly price	Search visibility	Booking capacity	Commission discount
Starter (Free)	\$0	Basic listing; ranked below all paid plans; ~100 profile views/mo cap	Up to 5 active bookings/month	None (full standard rate)
Professional	\$29/mo \$295/yr	Standard priority above Starter; unlimited impressions	Unlimited concurrent bookings	–1% on standard-band jobs (4% effective)
Premium	\$59/mo \$599/yr	Featured top placement in category + area; unlimited impressions	Unlimited + priority job alerts	–2% on standard-band jobs (3% effective)

1.4.3 Value Exchange and Break-Even Analysis

Workers evaluate subscription economics by comparing monthly plan cost against incremental commission savings plus the estimated revenue uplift from improved visibility. A Professional subscriber completing fifteen standard-band jobs of two hundred dollars each generates three thousand dollars in gross volume. At five percent commission the fee would be one hundred fifty dollars; at the reduced four percent rate the fee is one hundred twenty dollars, yielding thirty dollars in monthly commission savings. Against a twenty-nine dollar subscription, the plan breaks even before accounting for any additional bookings attracted through higher search ranking. A Premium subscriber completing the same volume saves sixty dollars in commission (three percent versus five percent), comfortably exceeding the fifty-nine dollar monthly fee while additionally benefiting from featured placement that industry benchmarks suggest can increase inbound enquiries by twenty to forty percent for service marketplaces.

From the platform operator's perspective, the dual model diversifies revenue risk. Transaction commissions scale automatically with marketplace activity and require no separate billing relationship, while subscriptions provide monthly recurring revenue that partially offsets fixed infrastructure costs—hosting, customer support, compliance, and payment processor minimums—even during seasonal troughs in job volume. The following illustrative projection assumes five hundred registered workers, of whom sixty percent remain on Starter, twenty-five percent subscribe to Professional, and fifteen percent subscribe to Premium, with one thousand completed standard-band jobs per month at an average gross value of one hundred fifty dollars.

Revenue source	Assumption	Monthly estimate
Transaction	1,000 jobs × \$150 avg × 5% blended	\$7,500

commissions	take-rate	
Professional subscriptions	125 workers × \$29/month	\$3,625
Premium subscriptions	75 workers × \$59/month	\$4,425
Total projected MRR	Commissions + subscriptions (illustrative)	\$15,550
Transaction commissions ~48% of illustrative MRR Variable revenue scaling with completed job volume. Collected at escrow release.		Worker subscriptions ~52% of illustrative MRR Recurring MRR from Professional and Premium plans. Predictable baseline income.

Break-even analysis is presented at two levels: worker subscription economics (when a paid plan pays for itself through commission savings) and platform operating economics (when total monthly revenue covers fixed and variable operating costs). Both use transparent formulas that can be validated in a spreadsheet or extended in production financial reporting.

1.4.3.1 Worker Subscription Break-Even

For a subscribed worker, monthly commission savings equal the number of completed standard-band jobs multiplied by the gross job value multiplied by the commission discount percentage. The subscription breaks even when savings equal the plan price:

Jobs to break even = Monthly subscription ÷ (Average job value × Commission discount %).

Plan	Monthly fee	Commission discount	Avg job \$150	Avg job \$200	Avg job \$250
Professional	\$29/mo	1%	19 jobs	14 jobs	12 jobs
Premium	\$59/mo	2%	20 jobs	15 jobs	12 jobs

At the illustrative two-hundred-dollar average job used earlier, Professional breaks even at 14 completed jobs per month and Premium at 15 jobs—before counting any uplift from higher search ranking or featured placement. Figure 6 visualises these thresholds across job sizes.

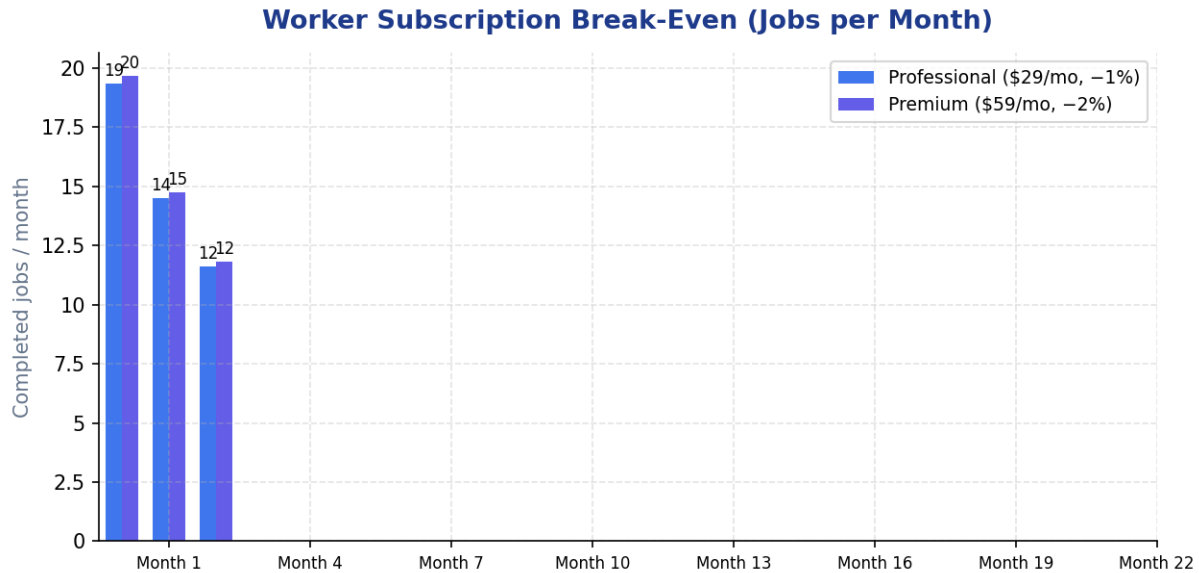


Figure 6 — Worker subscription break-even: completed jobs per month required to offset plan cost via commission savings.

1.4.3.2 Platform Operating Break-Even

From the operator's perspective, monthly profit equals total MRR minus operating costs. Fixed costs follow a bootstrap phased schedule (Bootstrap scenario — Months 1–6: \$3,200 · 7–12: \$3,800 · 13–24: \$4,500) reflecting founder-led launch before full-scale hiring. Variable costs are estimated at 2% of gross merchandise volume (GMV) to cover payment rails, per-job support, and dispute handling. Total monthly cost therefore equals:

Total cost = Fixed monthly costs + (Variable rate × Monthly GMV).

Platform break-even occurs in the first month where Total MRR $\geq$ Total cost. Applying the twenty-four-month projection model from Section 1.4.6 yields the following illustrative result:

Metric	Formula / assumption	Illustrative result
Fixed monthly costs	Phased: lean launch → growth → scale	Bootstrap scenario — Months 1–6: \$3,200 · 7–12: \$3,800 · 13–24: \$4,500
Variable cost rate	Payment processing + support allocation on GMV	2% of GMV
Break-even month	First month where MRR $\geq$ total operating cost	Month 25
Month 24 MRR	Commission + subscriptions (projected)	\$5,203

Month 24 operating cost	Fixed + variable on projected GMV	\$5,230
Month 24 net margin	MRR – operating cost	\$-27

Figure 7 plots projected MRR against total operating cost across twenty-four months. The intersection identifies the break-even point; shaded regions indicate cumulative operating deficit (before break-even) and surplus (after break-even). This complements the revenue growth charts in Section 1.4.6 by answering when the marketplace becomes self-sustaining under stated assumptions.

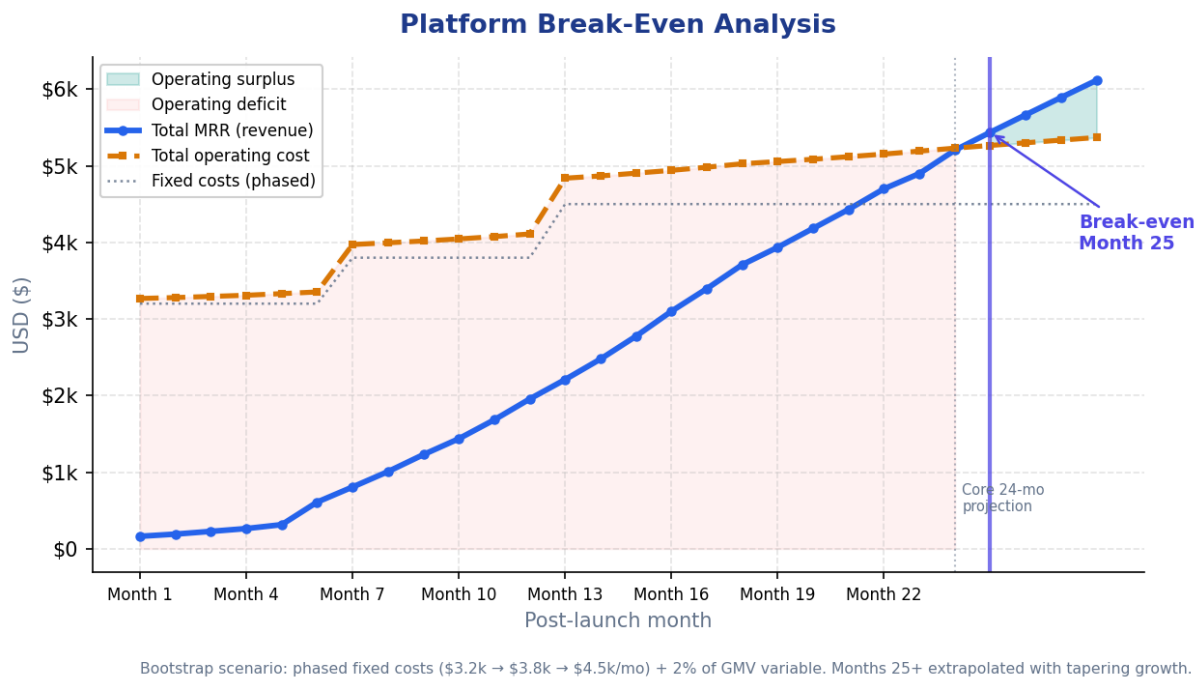


Figure 7 — Platform break-even: monthly revenue (MRR) versus total operating cost (fixed + variable).

1.4.4 Alignment with the ProTech Implementation

The transaction commission component is not merely theoretical: it is implemented in the escrow release path of the application. The `escrow/services.py` module calculates `platform_fee` as a percentage of the held amount, stores it on the `EscrowHold` record, and pays `worker_payout` net of that fee. The wallet and transaction tables provide the audit trail required for financial reporting and tax documentation in a commercial deployment. Extending the prototype to support tiered commission bands would require comparing `booking.price` against configured thresholds at release time, a straightforward enhancement to the existing service function without architectural change.

Worker subscription tiers represent the next planned product increment. Implementation would introduce a `WorkerSubscription` model linked one-to-one with `Worker` profiles, storing plan type, billing period, renewal date, and Stripe customer identifier. Search ranking in `browse_workers` would order results by plan priority before rating, and a monthly booking counter would enforce Starter-tier capacity limits. Django Admin would manage plan overrides for promotional trials. Together, the live

commission logic and the specified subscription framework demonstrate that ProTech is designed not only as an academic exercise but as a commercially viable marketplace with a clear path from prototype to revenue-generating production.

1.4.5 Go-to-Market and Growth Considerations

Launch sequencing should prioritise client liquidity before aggressive worker monetisation. An initial six-month worker acquisition phase may waive subscription fees entirely while keeping the five percent commission, allowing supply to accumulate without upfront barrier. Once search results contain sufficient choice for clients in core geographies, Professional and Premium plans activate with a thirty-day free trial for workers who complete at least three jobs, ensuring subscribers understand platform value before the first invoice. Referral credits—one month of Professional for each worker referred who completes five jobs—accelerate viral supply growth at lower paid acquisition cost than digital advertising alone.

Regulatory and ethical constraints require transparent fee disclosure at booking confirmation and on worker payout receipts, matching the clarity already shown on the escrow payment screen in the application. Subscription auto-renewal must comply with consumer protection rules in the operating jurisdiction, including easy cancellation from the worker settings page and pro-rata refunds where mandated. These considerations complete a business model that is financially sound, technically grounded in the existing codebase, and appropriate for presentation during the IT Innovation in Business project defence.

1.4.6 Growth Projections: Traffic and Economics

The following projections illustrate a plausible twenty-four-month evolution of ProTech following a controlled city-level launch. Figures are illustrative rather than audited forecasts; they are derived from a deterministic compound-growth model that assumes an initial six-month supply-building phase, twelve months of accelerating network effects as client choice and worker density improve conversion, and a final six-month maturity phase in which growth rates taper but absolute volumes continue to rise. The model aligns with the hybrid monetisation strategy described above: commission revenue scales with completed job volume while subscription revenue grows as a share of the worker base once paid plans activate in month seven.

Traffic growth is expressed through monthly unique visitors, monthly active users (MAU), and total page views. MAU is modelled at approximately sixty-two percent of visitor volume, reflecting repeat usage by clients checking booking status and workers managing job queues. Page views average three point six per visitor as users browse worker profiles, compare ratings, and navigate dashboard workflows.

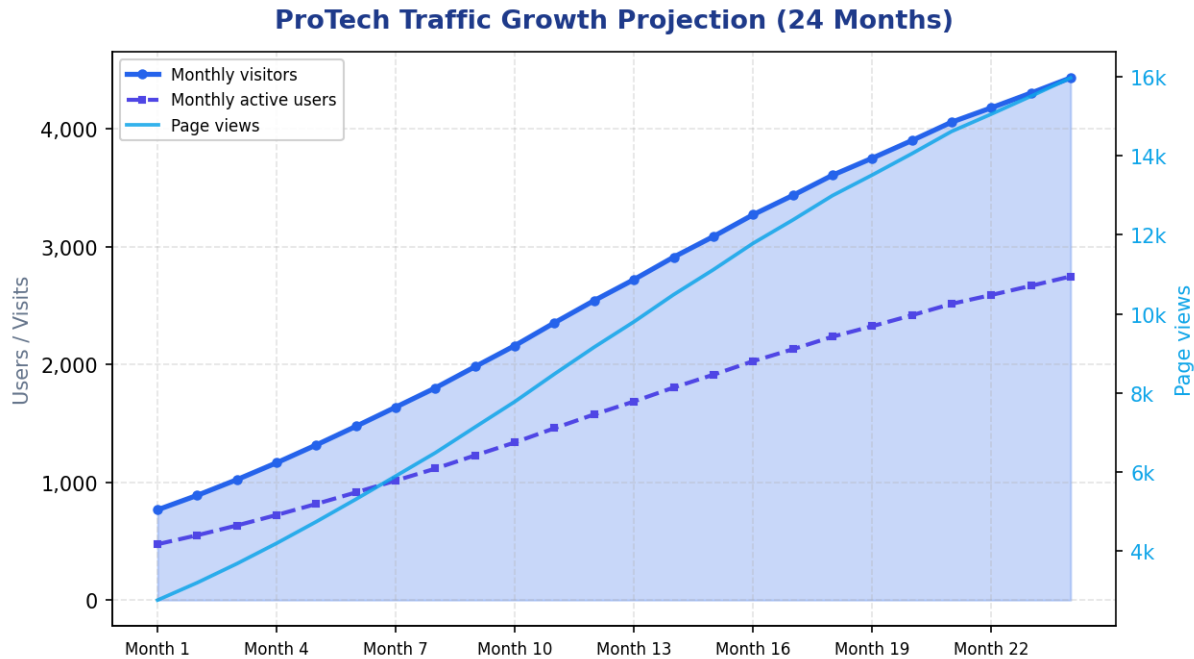


Figure 1 — Projected monthly visitors, active users, and page views (Months 1–24 post-launch).

Registered account growth follows a two-sided pattern characteristic of marketplaces. Client acquisition outpaces worker acquisition in absolute terms because the addressable client population is larger and onboarding friction is lower (no subscription decision). Worker growth is deliberately moderated early to preserve service quality and avoid empty search results in categories with insufficient geographic coverage. By month twenty-four, the model reaches approximately 9,500 client accounts and 1,100 worker accounts, a ratio that supports healthy liquidity in a mid-sized metropolitan market.

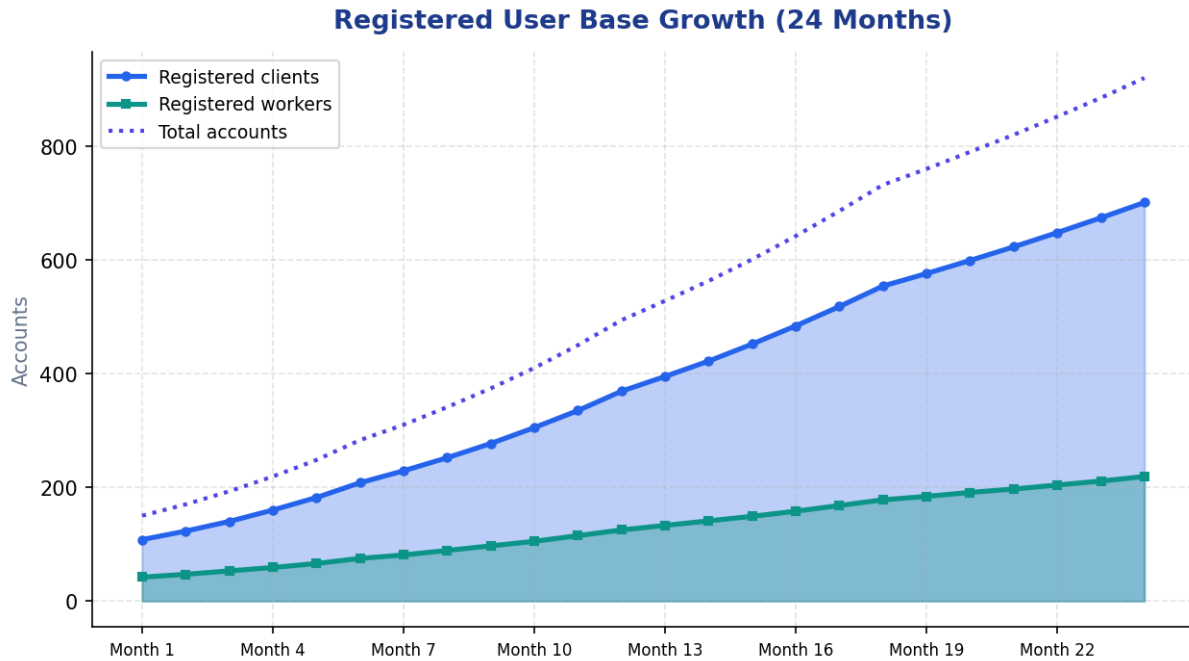


Figure 2 — Projected growth of registered client and worker accounts.

Period	Visitors	Clients	Workers	Jobs / mo	MRR
Month 6	1,476.2248 47552	208	75	53	\$609
Month 12	2,544.1201 13698246	369	125	100	\$1,955
Month 18	3,608.2406 551362005	554	178	159	\$3,709
Month 24	4,435.1385 10870497	701	219	207	\$5,203

Economic growth is driven by two coupled variables: gross merchandise volume (GMV), defined as the total value of completed jobs processed through escrow, and monthly recurring revenue (MRR), combining transaction commissions and worker subscriptions. Completed bookings grow from twenty-two jobs in month one to approximately two thousand two hundred jobs by month twenty-four, while average job value rises modestly from one hundred thirty-five to one hundred eighty dollars as the platform attracts a higher proportion of multi-hour commercial engagements. Commission MRR therefore expands both through job count and through GMV per job.

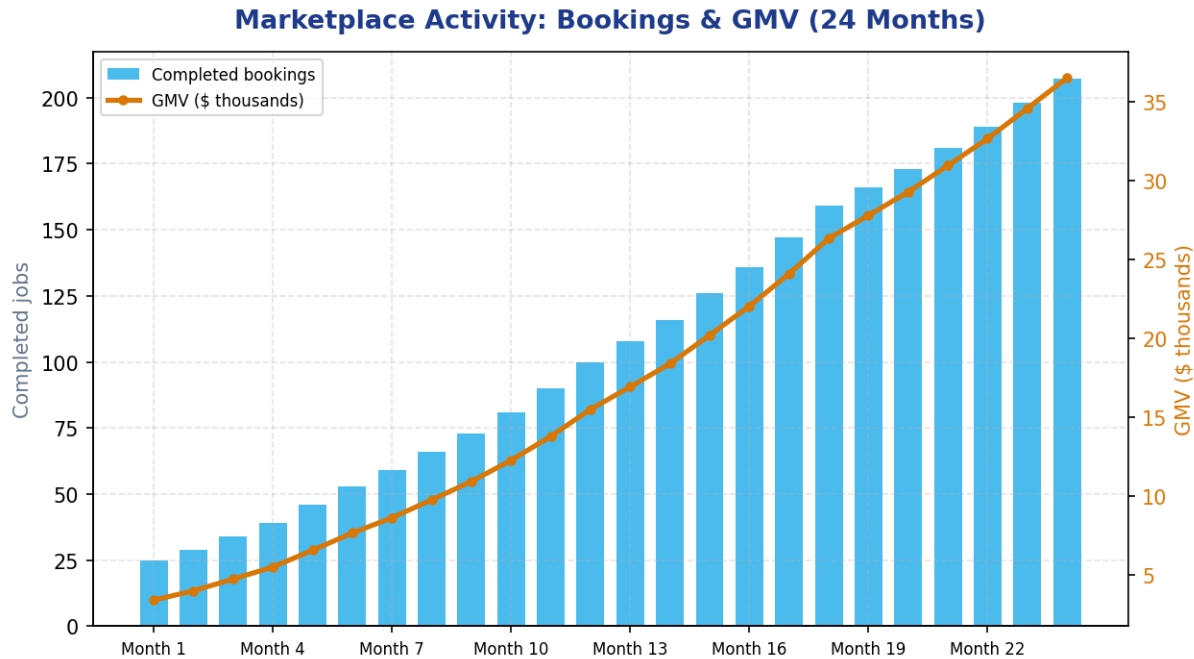


Figure 3 — Completed bookings (bars) and gross merchandise volume in \$ thousands (line).

Total MRR crosses fifteen thousand dollars around month twelve—consistent with the steady-state snapshot in Section 1.4.3—and approaches fifty thousand dollars by month twenty-four. Subscription revenue, which is zero during the initial promotional waiver period, rises to represent roughly thirty-nine percent of MRR by the end of year two as worker penetration into Professional and Premium tiers increases. The stacked revenue chart below separates commission and subscription contributions month by month.

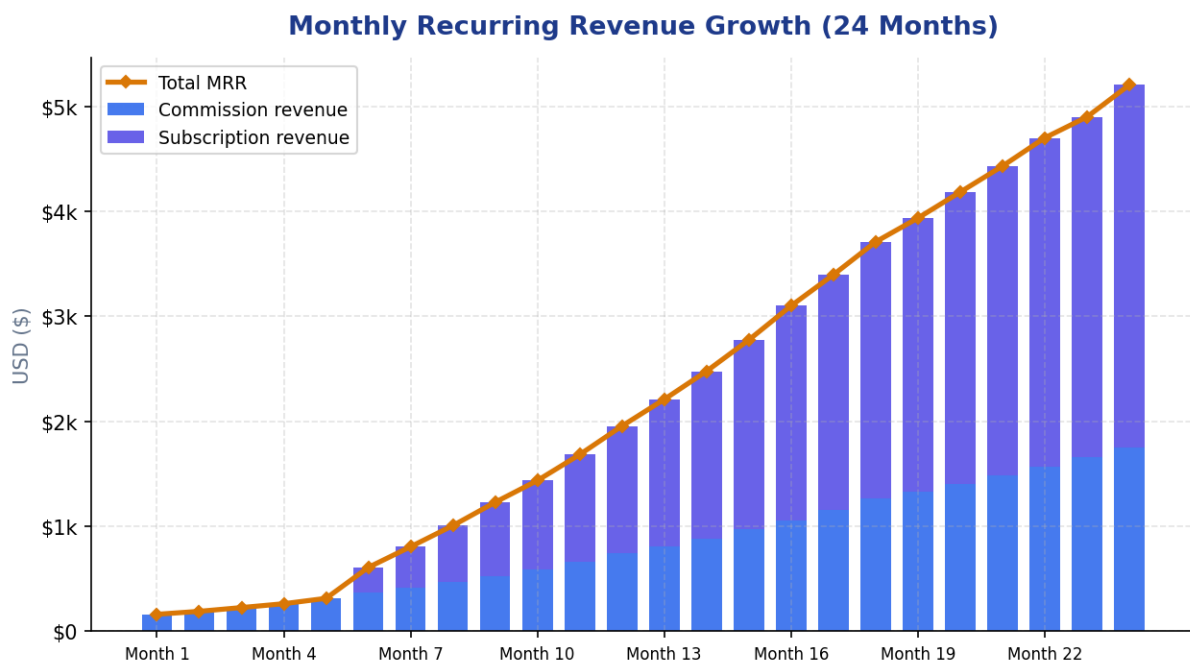


Figure 4 — Monthly recurring revenue: commission (blue) + subscriptions (indigo); total MRR (gold line).

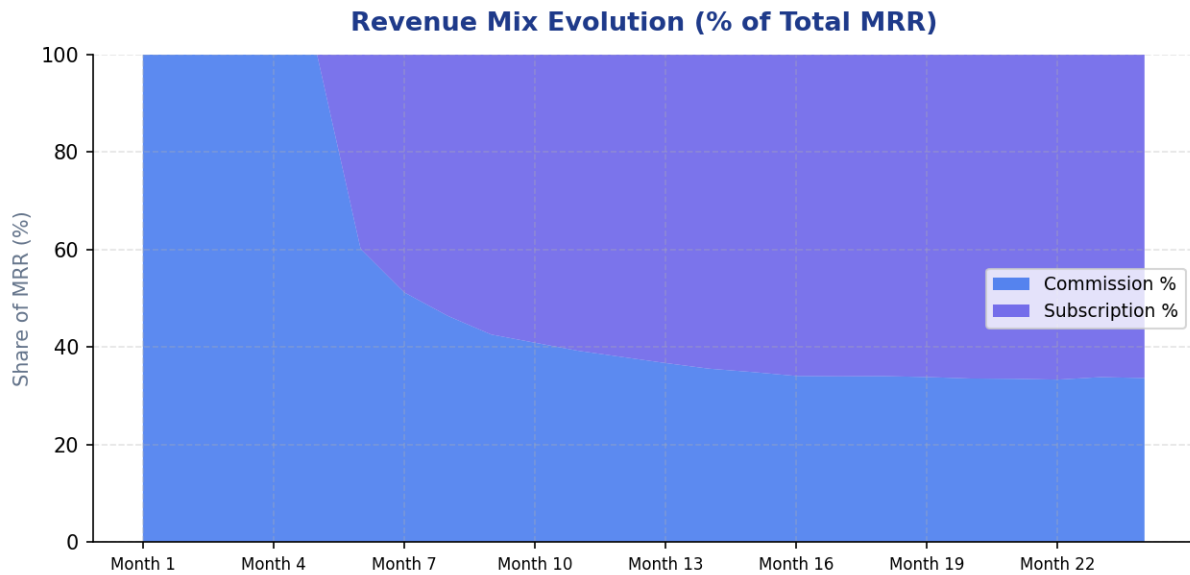


Figure 5 — Evolution of revenue mix: subscription share of MRR increases after paid plans launch (Month 7).

Three takeaways from these projections stand out for the project defence. Traffic and revenue feed each other: more visitors mean more bookings and commission income, which can fund marketing that brings more visitors. Subscription revenue steadies cash flow when job volume dips in winter. The Django and Redis stack can scale with more Gunicorn workers, PostgreSQL read replicas, and Redis partitioning without a full rewrite, so ProTech is built to grow beyond the academic demo.

2. System Design

2.1 Functional Architecture

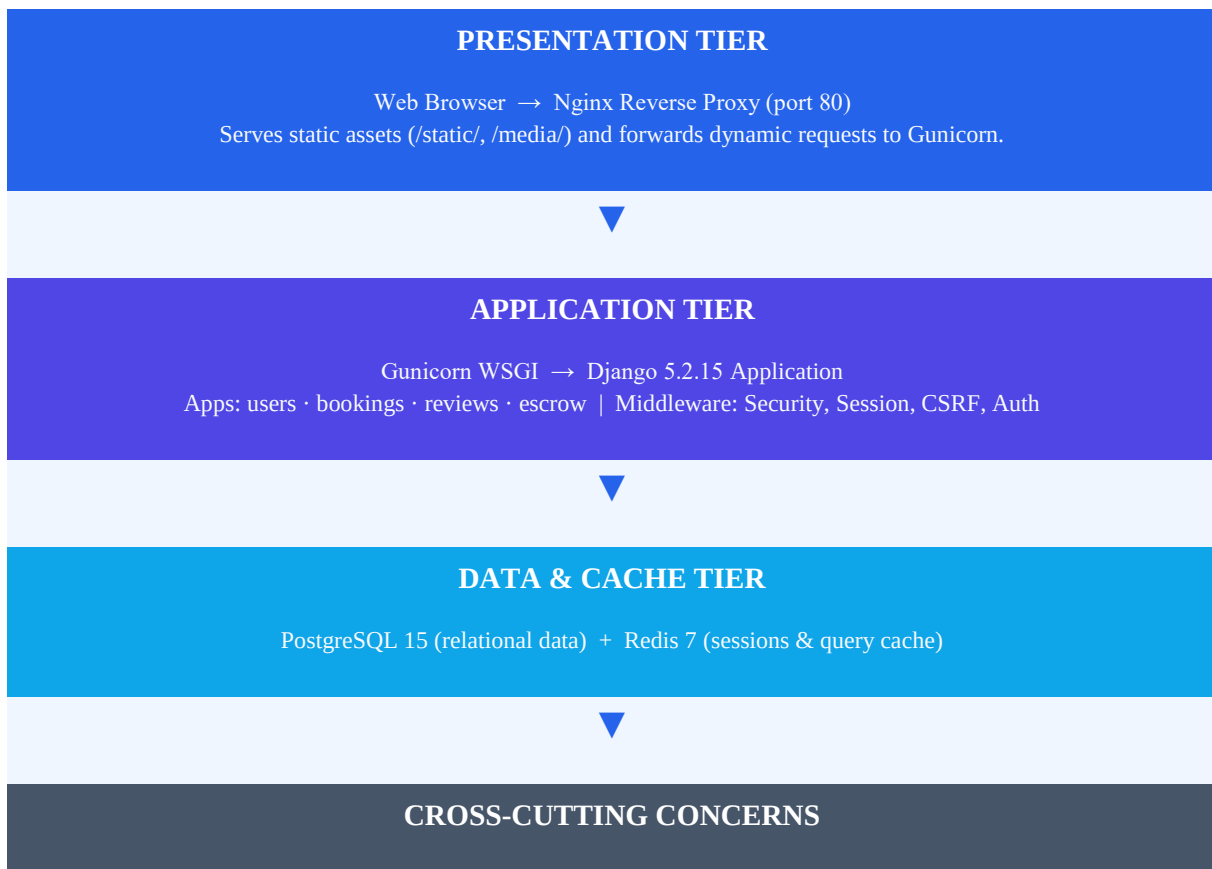
ProTech adopts a three-tier architecture adapted for cloud-native container deployment. The presentation tier consists of HTML, CSS, and JavaScript templates rendered server-side by Django using a Jinja2 template backend, ensuring fast initial page loads and strong SEO characteristics without requiring a separate single-page application build pipeline.

The application tier is implemented as a Django 5.2.15 project partitioned into four domain applications: users (identity, profiles, messaging, notifications), bookings (scheduling and status workflow), reviews (ratings and reputation), and escrow (wallet and payment holds). Gunicorn serves the WSGI application with multiple worker processes; Nginx terminates HTTP on port 80 and proxies dynamic traffic to Gunicorn while serving static and media files from shared Docker volumes.

PostgreSQL 15 stores relational data; Redis 7 handles sessions and query caching. Django middleware checks security headers, sessions, CSRF tokens, and authentication on every request before any view runs. If authentication fails, the request is blocked rather than allowed through.

2.2 Architecture Diagram

The diagram below shows how a request moves through the ProTech stack. Each layer runs in its own Docker container, which keeps development, demo, and production deployments consistent.



Structured logging · RBAC decorators · Escrow ledger · Django Admin CMS



DEPLOYMENT

Docker Compose: db · redis · backend · nginx | CI: GitHub Actions

2.3 Use Cases

The platform supports eleven primary use cases spanning registration, authentication, marketplace discovery, financial workflow, and administration. The table below summarises each actor interaction; full behavioural detail is implemented in the corresponding Django views and service modules.

ID	Use Case	Description
UC-01	Register as Client	A visitor creates a client account with name, email, phone, and password, then signs in to access the client dashboard.
UC-02	Register as Worker	A professional registers with skills, service area, and experience; the system creates a linked Worker profile.
UC-03	Login and Logout	Users authenticate with email and password via a custom EmailBackend; sessions are stored in Redis in production.
UC-04	Reset Password	Users request a secure reset link; Django's PasswordResetView delivers the token (console email in development).
UC-05	Browse Workers	Clients search by skill, location, and minimum rating; results are served from Redis cache when available.
UC-06	Book a Service	A client selects a worker and submits schedule, address, description, and price to create a pending booking.
UC-07	Manage Bookings	Both parties advance status through pending, confirmed, in progress, completed, or cancelled states.
UC-08	Fund Escrow	The client transfers wallet balance into platform escrow for a specific booking prior to work commencing.
UC-09	Release or Refund Escrow	On completion, funds release to the worker minus platform fee; on cancellation, funds return to the client.

UC-10	Leave Review	After completion, the client submits a star rating and comment; the worker's aggregate rating is recalculated.
UC-11	Admin Management	Staff operate Django Admin to create, read, update, and delete users, categories, bookings, and ledger records.

2.4 Key Features and Data Model

Role-based access control is enforced through a custom `user_type` field on the extended User model and a `@user_type_required` decorator that redirects unauthorised role access before protected views render. Clients interact with `browse_workers`, `favorites`, and booking creation flows; workers interact with job acceptance, status updates, and wallet receipt flows. Superusers access Django Admin for content management without custom CMS development overhead.

The relational schema centres on User and Worker profiles, ServiceCategory taxonomy, Booking records linking clients to workers, Review entities tied one-to-one to completed bookings, and an escrow ledger comprising UserWallet balances, EscrowHold records per booking, EscrowVault aggregates, and WalletTransaction audit rows. Supplementary Message, Notification, Favorite, and WorkerAvailability models support engagement features that differentiate the platform from a minimal booking form.

The escrow payment workflow is central to client trust on the platform. The following process diagram summarises how funds move between the client wallet, the platform vault, and the worker wallet during a typical job lifecycle.

Step	Stage	Description
1	Client deposits demo funds	UserWallet.available_balance increases via <code>deposit_demo_funds()</code> .
2	Client books a worker	Booking record created with status pending and <code>payment_status</code> pending.
3	Client funds escrow	Wallet debited; EscrowHold created; <code>vault.total_held</code> updated.
4	Worker completes job	Worker marks booking complete; <code>release_escrow()</code> runs automatically.
5	Payout & fee	Worker receives amount minus platform fee (default 5%); transaction logged.
6	Cancellation path	If cancelled while held, <code>refund_escrow()</code> returns full amount to client wallet.

Booking records progress through a finite state machine shared by clients and workers. The diagram below shows permitted transitions; escrow funding and release/refund hooks are tied to `payment_status` alongside operational status.

State	Who acts	Next states	Escrow interaction
pending	Worker	confirmed · cancelled	Client may fund escrow while pending.
confirmed	Worker	in_progress · cancelled	Escrow held if previously funded.
in_progress	Worker	completed · cancelled	Release on complete; refund on cancel.
completed	Client	— (terminal)	<code>release_escrow()</code> pays worker minus platform fee.
cancelled	Either party	— (terminal)	<code>refund_escrow()</code> if funds were held.

2.5 Entity-Relationship Model

The diagram below shows the core relational schema implemented across the users, bookings, reviews, and escrow Django applications. User is extended from Django's `AbstractUser` with a `user_type` discriminator. Worker maintains a one-to-one link to User for professionals. Booking is the central transactional entity, connecting a client (User) to a Worker and optional `ServiceCategory`. Review and `EscrowHold` each attach one-to-one to a completed or funded booking. `UserWallet`, `EscrowVault`, and `WalletTransaction` form the internal financial ledger.

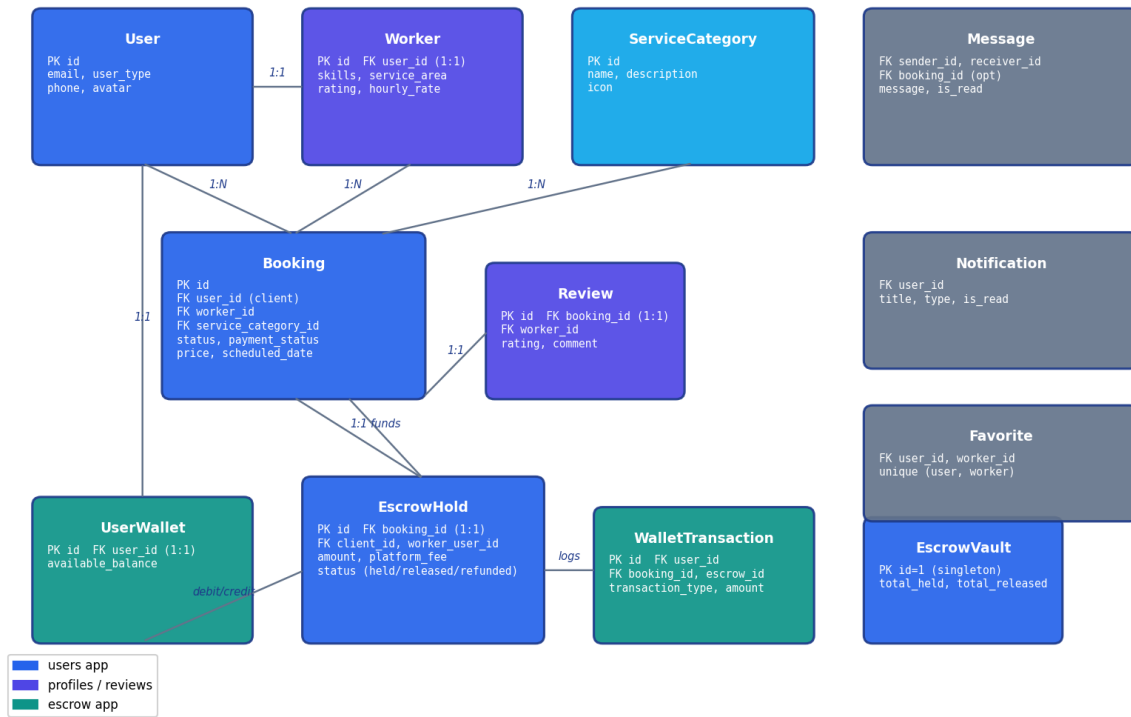
ProTech Entity-Relationship Diagram (Core Schema)

Figure — Entity-relationship diagram of core ProTech database models.

2.6 Escrow Sequence Diagram

While the escrow workflow table in Section 2.4 describes stages in prose, the sequence diagram below shows the runtime interaction between the client browser, Django views, escrow/services.py, and PostgreSQL during a typical fund-and-release cycle. Each monetary step executes inside a database transaction; Decimal arithmetic prevents floating-point rounding errors in the ledger.

Escrow Payment Sequence Diagram

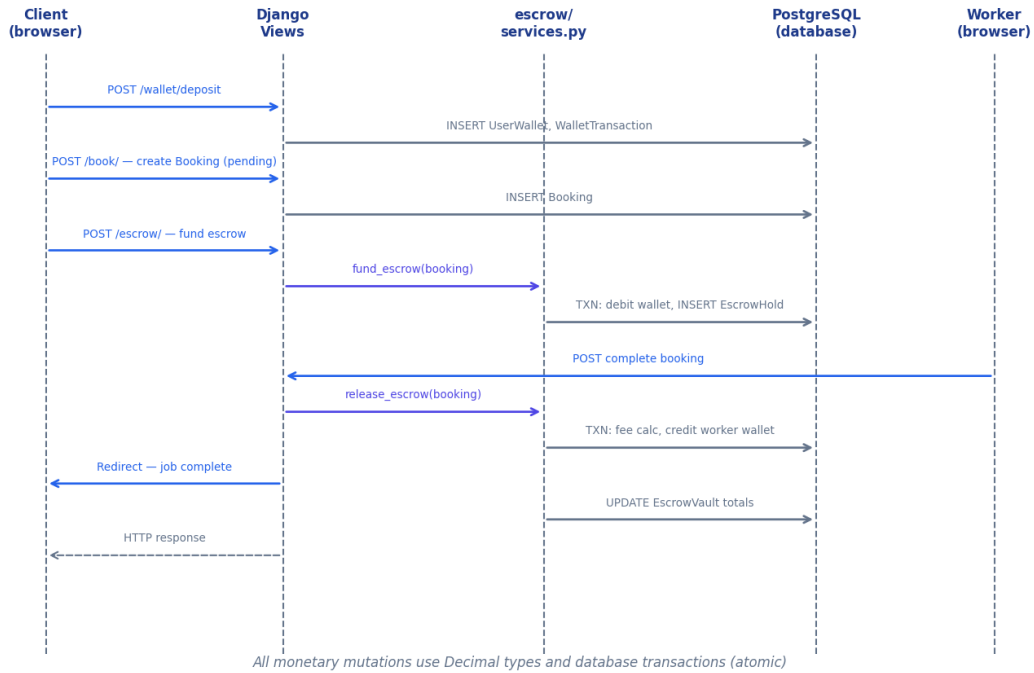


Figure — Sequence diagram: client deposit, booking, escrow funding, and payout on completion.

3. Implementation

3.1 Technology Choices

Technology selection balanced academic requirements, developer productivity, and production viability. Django was chosen over lighter micro-frameworks because its built-in admin, ORM, authentication, CSRF middleware, and migration system reduce boilerplate while demonstrating enterprise patterns expected in the module rubric. PostgreSQL provides ACID guarantees essential for booking and escrow integrity. Redis addresses session latency and repeat worker-list queries under concurrent load.

Component	Technology	Rationale
Back-end framework	Django 5.2.15	Integrated ORM, admin CMS, security middleware, and mature deployment documentation.
API readiness	Django REST Framework 3.17	Session-authenticated API layer prepared for future mobile or SPA clients.
Front-end	Django Templates (Jinja2)	Server-rendered pages reusing the existing ProTech HTML/CSS design system.
Database	PostgreSQL 15	Relational integrity, indexing on bookings and messages, Docker health checks.
Cache / sessions	Redis 7 + django-redis	Sub-millisecond cache reads; production session backend with namespaced keys.
App server	Gunicorn 26	Multi-worker WSGI process model suitable for CPU-bound Django view rendering.
Reverse proxy	Nginx Alpine	Static file offload, gzip-ready proxy headers, single public port 80.
Containers	Docker Compose 3.8	Reproducible four-service stack for local demo and cloud deployment.

3.2 Application Structure

Source code is organised into Django applications that mirror business domains rather than technical layers, which keeps view functions discoverable and allows each app to own its models, admin registrations, and tests. The `protech_project` package holds cross-cutting configuration: settings with environment-driven secrets, URL routing that preserves legacy endpoint names for template

compatibility, a Jinja2 environment shim exposing `url_for` and `csrf_field` helpers, cache utilities in `cache_utils.py`, and a JSON health endpoint for monitoring.

The users application implements authentication views, client and worker dashboards, profile and settings pages, messaging, notifications, and favorites. The bookings application owns creation, listing, and POST-driven status transitions including integration hooks into escrow release on completion. The reviews application handles post-job rating submission and cache invalidation when worker aggregates change. The escrow application encapsulates all monetary logic in `services.py`, keeping views thin and financial rules testable in isolation.

3.3 Security, Authentication, and Escrow

Authentication combines Django's session framework with a custom EmailBackend permitting login forms to accept email addresses as identifiers, matching user expectations set by consumer marketplace applications. Passwords are hashed with PBKDF2 through Django's default validators prohibiting common and entirely numeric passwords. Registration branches on form submission type to create either a client User or a User paired with a Worker row. Password reset reuses Django's tokenised email flow; development environments print reset URLs to the console while production can enable SMTP through environment variables without code changes.

Security controls address the OWASP-aligned requirements of the project brief. `CsrfViewMiddleware` validates tokens on every POST request; templates include `csrf_field()` on all mutating forms. The ORM parameterises queries, eliminating raw SQL injection surfaces in application code. Template auto-escaping mitigates reflected XSS; production settings enable HSTS, secure cookies, SSL redirect, and X-Frame-Options denial. Secrets including `SECRET_KEY` and database credentials reside exclusively in `.env` files excluded from version control via `.gitignore`, with separate `.env.dev.example` and `.env.prod.example` templates documenting required variables.

The escrow subsystem implements an internal ledger in `escrow/services.py`. Clients deposit demonstration funds into `UserWallet` rows, transfer booking amounts into `EscrowHold` records, and trigger release or refund operations that update `WalletTransaction` audit trails and `EscrowVault` aggregates. A configurable platform fee (`ESCROW_PLATFORM_FEE_PERCENT`, default five percent) is deducted on worker payout. All mutations execute inside database transactions with Decimal quantisation to prevent floating-point rounding errors. The service interface is intentionally structured so a production deployment could swap the ledger backend for Stripe Payment Intents and Connect webhooks without rewriting view layers.

3.4 Caching and Logging

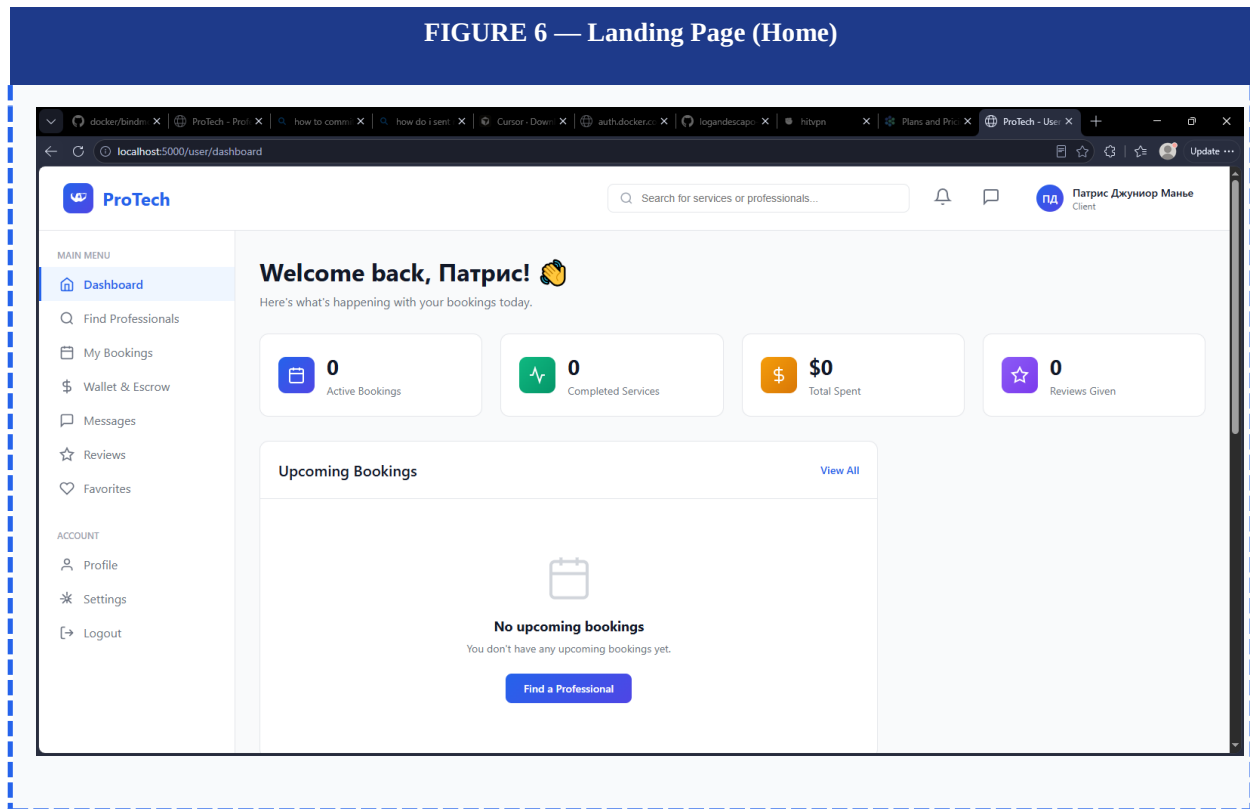
Caching is implemented in `protech_project/cache_utils.py` using Django's cache framework backed by Redis in production. Worker search results are cached for three hundred seconds keyed by a hash of filter parameters; service category lists cache for six hundred seconds. When a new review alters worker rating aggregates, `invalidate_workers_cache()` clears stale entries. Continuous integration sets `USE_LOCMEM_CACHE` so automated tests execute without a live Redis daemon, demonstrating environment-aware configuration rather than hard-coded infrastructure assumptions.

Logging configuration in settings.py defines verbose formatters writing simultaneously to stdout and logs/django.log. Dedicated loggers for the users, bookings, and escrow applications allow operators to tune verbosity per domain via the DJANGO_LOG_LEVEL environment variable. View and service modules emit INFO records for successful authentication and booking transitions, WARNING records for failed login attempts, and ERROR records for escrow exceptions, supporting post-incident diagnosis during project defence questioning.

3.5 Application Interface (Screenshot Placements)

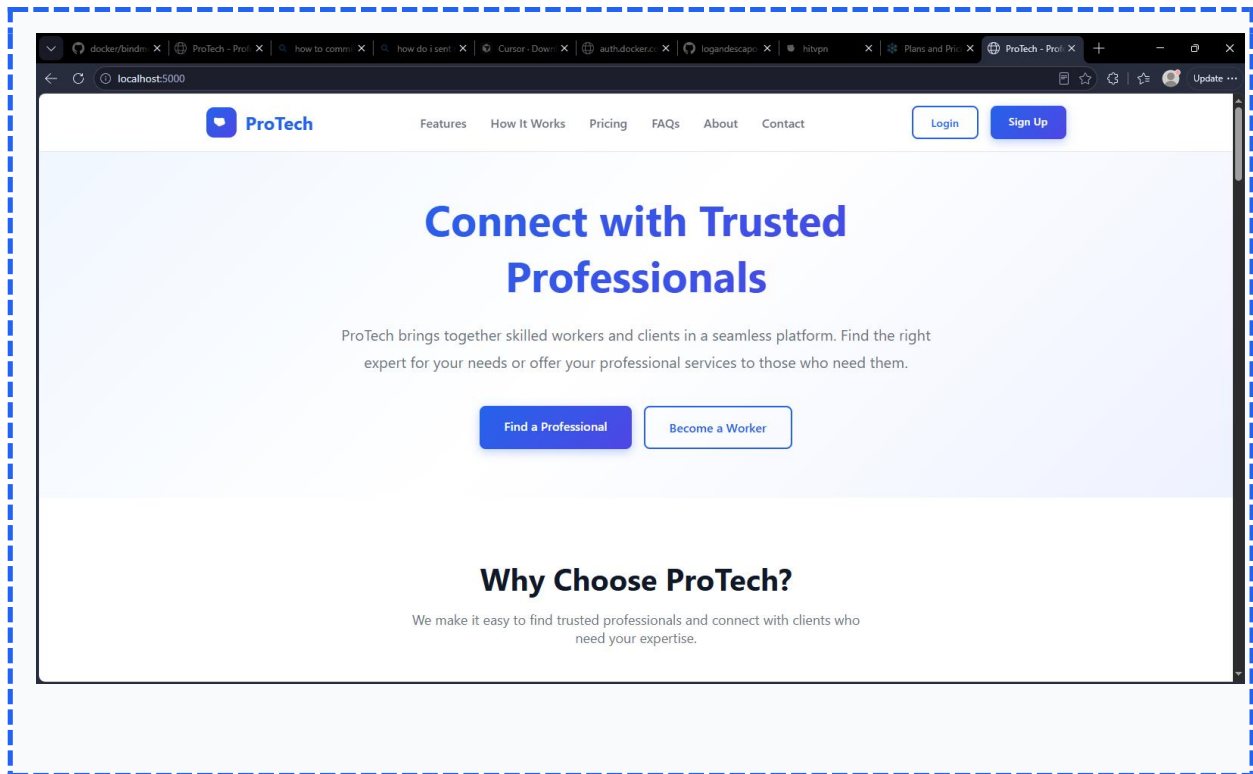
The ProTech interface uses server-rendered Django templates with responsive HTML and CSS. Screenshots in this report show that the application works as described. The three placeholders below cover the main entry points; Appendix C lists fourteen more. To capture your own, run docker compose up, open <http://localhost>, log in with the demo credentials in Appendix B, and save PNG images at 1280×720 or higher.

FIGURE 6 — Landing Page (Home)



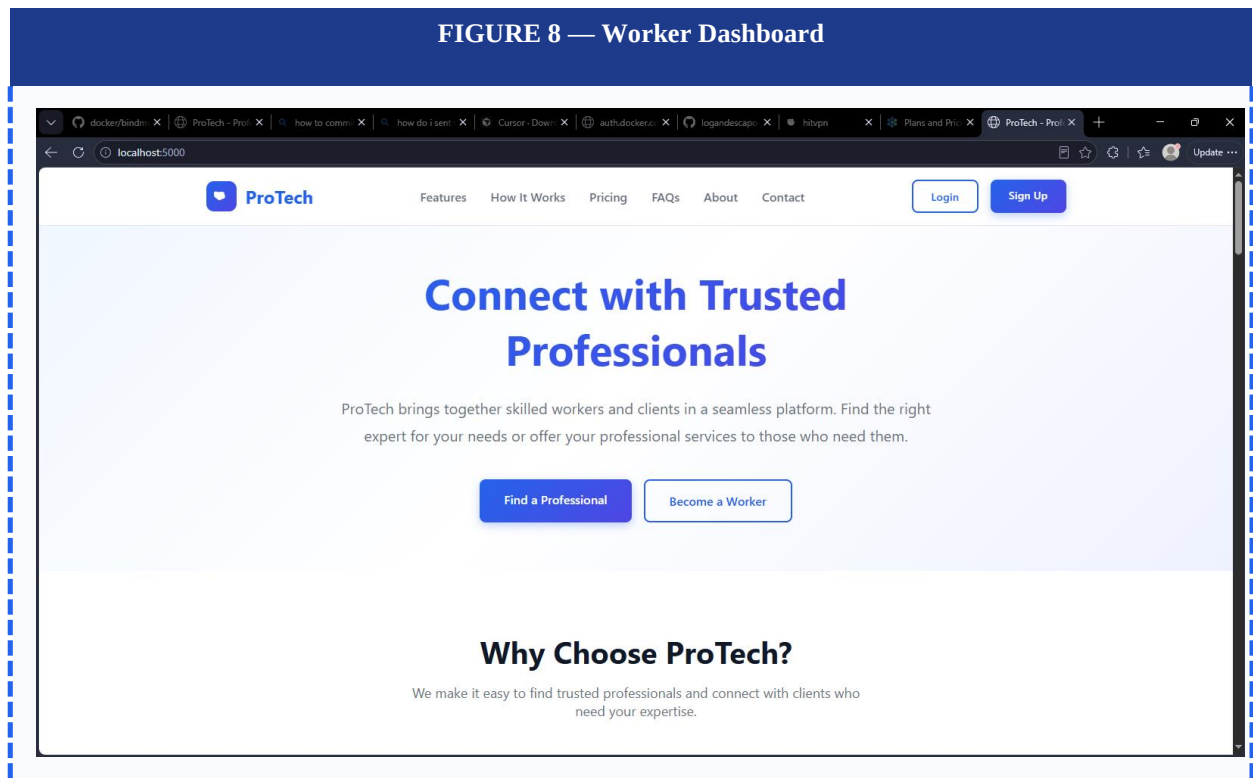
Full-width hero, feature highlights, and primary Sign Up / Find a Professional CTAs.

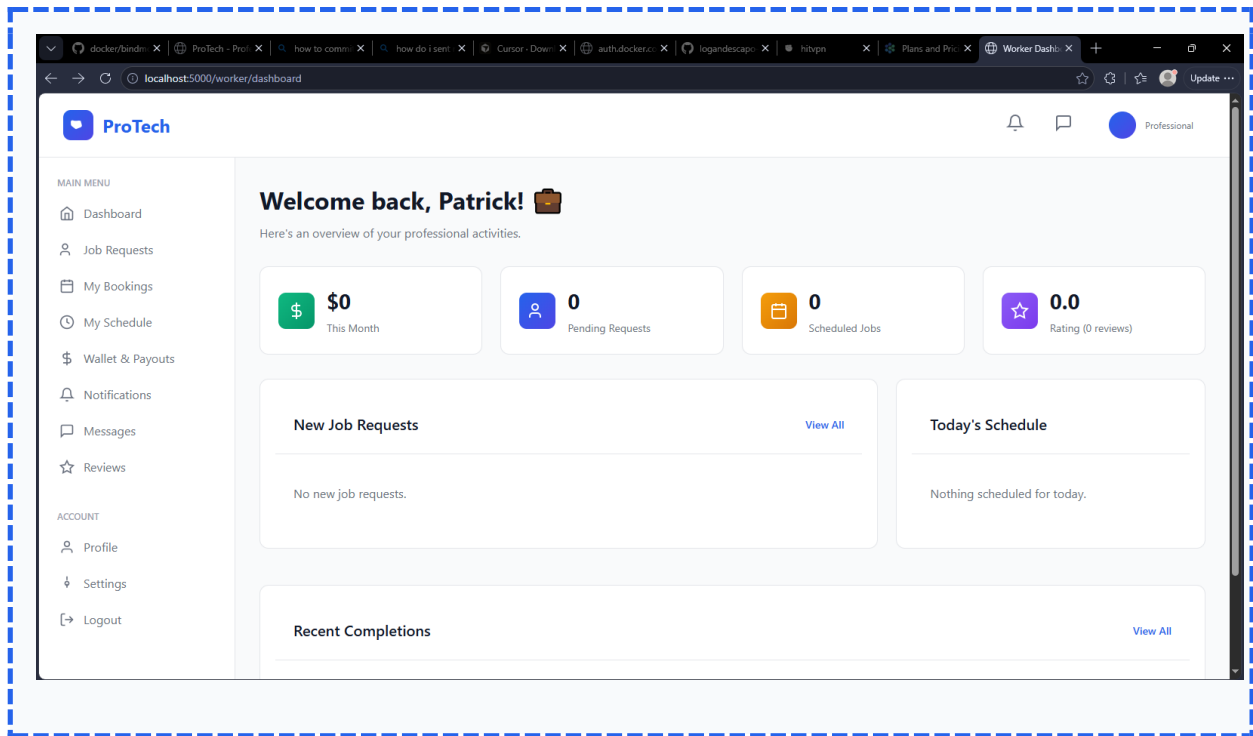
FIGURE 7 — Client Dashboard



Upcoming bookings, quick actions, sidebar navigation, and notification badges.

FIGURE 8 — Worker Dashboard





Pending requests, today's schedule, earnings summary, and job actions.

Screenshots for booking, escrow, wallet, reviews, messaging, and Django Admin are in Appendix C. Click inside each dashed box in Word, then Insert → Pictures, or drag your image onto the placeholder.

4. Testing and Deployment

Quality assurance follows a pragmatic test pyramid. Unit tests in each Django application validate model constraints, password hashing, and relationship integrity. Integration tests in `users/tests_integration.py` exercise the health endpoint, email login redirect, role-based access denial, and password reset page availability. Escrow integration tests confirm that funding a booking correctly debits the client wallet and sets `payment_status` to `escrow_held`. At the time of writing, seventeen automated tests pass when executed via `python manage.py test` with `USE_LOCMEM_CACHE` enabled.

GitHub Actions workflow `ci.yml` runs on every push to the main branch. The pipeline installs Python dependencies, executes flake8 critical-error checks, applies migrations against an ephemeral PostgreSQL service container, runs the full Django test suite, and verifies the `/health/` endpoint responds with status ok. This continuous integration gate prevents regressions from reaching the deployment branch and provides evidence of professional software practice for academic assessment.

Local and demonstration deployment uses Docker Compose. The db service runs `postgres:15-alpine` with volume-backed persistence and `pg_isready` health checks. The redis service provides cache and session storage. The backend service builds from `Dockerfile.backend` and executes `docker-entrypoint.sh`, which runs migrations, collects static files, seeds demonstration accounts when the database is empty, and starts Gunicorn. The nginx service listens on port eighty, proxies application traffic, and serves staticfiles and media volumes directly for efficiency.

Service	Image / Build	Port	Responsibility
db	postgres:15-alpine	5432	Primary relational datastore with health checks.
redis	redis:7-alpine	6379	Cache backend and production session store.
backend	Dockerfile.backend	8000	Django application via Gunicorn and entrypoint script.
nginx	nginx:alpine	80	Public HTTP entry point, reverse proxy, static file server.

An evaluator starts the complete stack with `docker compose up --build` and accesses the application at `http://localhost`, the admin console at `http://localhost/admin`, and the health probe at `http://localhost/health/`. Cloud deployment options documented in the README include Railway and Render blueprints; production hardening requires `DEBUG=False`, strong `SECRET_KEY` rotation, `ALLOWED_HOSTS` restriction, HTTPS termination, and SMTP configuration for password reset delivery.

5. Conclusion, Limitations, and Literature

5.1 Conclusion

ProTech meets the full-stack website development brief with a working marketplace, two user roles, admin content management, escrow payments, Docker deployment, Redis caching, logging, and automated tests. Cloning the repo and running Docker Compose gives a fully interactive demo for the eight-to-ten-minute presentation and technical defence.

A few lessons came out of the build. Splitting Django apps by business area (users, bookings, escrow) stayed manageable as features grew; one big views file would not have. Moving from an earlier Flask prototype meant adding a Jinja2 layer so url_for and flash messages still worked — a reminder that framework changes cost real time. Escrow needed Decimal types and database transactions; floats would have caused rounding bugs. Running migrations and seed data in the Docker entrypoint made first-run much smoother for anyone testing the project.

Future development should integrate a licensed payment service provider for regulatory compliance, add WebSocket or Celery-driven real-time notifications, expand browser-based end-to-end tests with Selenium or Cypress, and optionally expose a React front end consuming Django REST Framework endpoints. Geolocation-aware worker matching, calendar synchronisation, and two-factor authentication represent further enhancements that would move the platform toward commercial viability.

5.2 Limitations

No academic or commercial software project is without constraints. ProTech is a functional prototype suitable for demonstration and defence, but several limitations should be stated clearly so that assessors can judge the work on realistic terms.

Limitation	Current state	Mitigation path
Payment processing	Demo wallet deposits only; no live card or bank integration	Stripe Payment Intents + Connect webhooks
Worker subscriptions	Business rules documented; billing not implemented in code	WorkerSubscription model + Stripe Billing
Geographic scope	Single-region demo; no geolocation matching	PostGIS or third-party geocoding API
Real-time features	Messaging is page-refresh based; no WebSockets	Django Channels or Celery notifications
Test coverage	17 integration/unit tests; no browser E2E suite	Selenium or Cypress against Docker stack

Flask legacy code	app.py retained for reference; Django is canonical	Remove legacy modules after full migration audit
Growth projections	Illustrative model, not audited market research	Pilot launch data to validate assumptions

5.3 Literature Review

The design of ProTech draws on established research and industry practice in platform economics, two-sided markets, web application architecture, and information security. This section situates the project within that body of knowledge rather than presenting it as an isolated technical exercise.

Two-sided marketplaces must solve the chicken-and-egg problem of attracting both demand (clients) and supply (workers) simultaneously. Rochet and Tirole (2003) formalise how platforms balance pricing on each side to maximise participation; Evans and Schmalensee (2016) extend this to modern digital matchmakers. ProTech's decision to keep client access free while monetising workers through commissions and optional subscriptions follows the asymmetrical pricing pattern common to services marketplaces such as Thumbtack and TaskRabbit, where supply-side professionals derive ongoing commercial value from lead generation and payment infrastructure.

Trust is a recurring theme in online service exchange. Edelman et al. (2012) demonstrate that reputation mechanisms significantly affect buyer willingness to transact; escrow and third-party payment holding further reduce perceived risk. ProTech combines star ratings tied to verified completed bookings with an internal escrow ledger, aligning with the trust-building patterns documented in gig-economy platform research. The review system enforces a one-review-per-booking constraint to limit grade inflation and spam.

On the technical side, Fielding's REST architectural style (2000) and Fowler's enterprise patterns (2002) informed the separation of domain applications and the thin-view / thick-service structure in escrow/services.py. Django's batteries-included philosophy matches the academic rubric requirement for integrated authentication, admin, ORM, and migration tooling without custom reinvention. Containerisation via Docker Compose reflects the DevOps practice of reproducible environments described in industry deployment guides (Docker Inc., 2024).

Security decisions reference the OWASP Top Ten (2021), which ranks broken access control, cryptographic failures, and injection among the highest risks for web applications. ProTech addresses these through RBAC decorators, PBKDF2 password hashing, ORM parameterisation, CSRF middleware, and template auto-escaping. Redis-backed caching follows performance guidance from the Redis documentation and django-redis integration patterns for session and query offload under concurrent read load.

5.4 References

The following sources were consulted during research, design, implementation, and documentation.

- [1] Django Software Foundation. (2024). Django Documentation (v5.2). <https://docs.djangoproject.com/>

- [2] Django REST Framework. (2024). API Guide and Authentication. <https://www.django-rest-framework.org/>
- [3] Docker Inc. (2024). Docker Compose Specification. <https://docs.docker.com/compose/>
- [4] Edelman, B., Luca, M., & Svirsky, D. (2012). From niches to riches: Anatomy of the long tail. Harvard Business School Working Paper (reputation in digital marketplaces).
- [5] Evans, D. S., & Schmalensee, R. (2016). Matchmakers: The New Economics of Multisided Platforms. Harvard Business Review Press.
- [6] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.
- [7] Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
- [8] Mozilla Developer Network. (2024). Web Security (CSRF, XSS). <https://developer.mozilla.org/en-US/docs/Web/Security>
- [9] Nginx Inc. (2024). NGINX Admin Guide. <https://nginx.org/en/docs/>
- [10] OWASP Foundation. (2021). OWASP Top Ten Web Application Security Risks. <https://owasp.org/www-project-top-ten/>
- [11] PostgreSQL Global Development Group. (2024). PostgreSQL 15 Documentation. <https://www.postgresql.org/docs/>
- [12] Redis Ltd. (2024). Redis Documentation. <https://redis.io/docs/>
- [13] Rochet, J.-C., & Tirole, J. (2003). Platform competition in two-sided markets. Journal of the European Economic Association, 1(4), 990–1029.

Appendix A — Project Structure

The repository root contains Django project configuration, application modules, deployment assets, and continuous integration definitions. Legacy Flask modules (app.py, auth.py) remain for reference but are superseded by the Django stack described in this report.

Path	Purpose
manage.py	Django management entry point for migrations, tests, and runserver.
protech_project/	Settings, URLs, Jinja2 environment, cache utilities, health view.
users/	Custom User model, authentication, dashboards, messaging, notifications.
bookings/	Booking creation, listing, and status workflow views.
reviews/	Post-job ratings and review listing.
escrow/	Wallet views and escrow/services.py ledger implementation.
templates/	Server-rendered HTML templates and registration email bodies.
nginx/nginx.conf	Reverse proxy and static file routing rules.
docker-compose.yml	Four-service orchestration manifest.
Dockerfile.backend	Production Django image build instructions.
docker-entrypoint.sh	Migrate, collectstatic, seed_demo, and Gunicorn startup.
.github/workflows/ci.yml	Automated lint, migrate, test, and health-check pipeline.

Appendix B — Demo Credentials

Demo accounts are created by python manage.py seed_demo or automatically on first Docker start. Use them to walk through client and worker flows during the defence without setting up data by hand.

Role	Email	Password	Dashboard URL
Client	john@example.com	password123	/user/dashboard/

Worker	mike@example.com	password123	/worker/dashboard/
Admin	(created via createsuperuser)	(chosen at setup)	/admin/

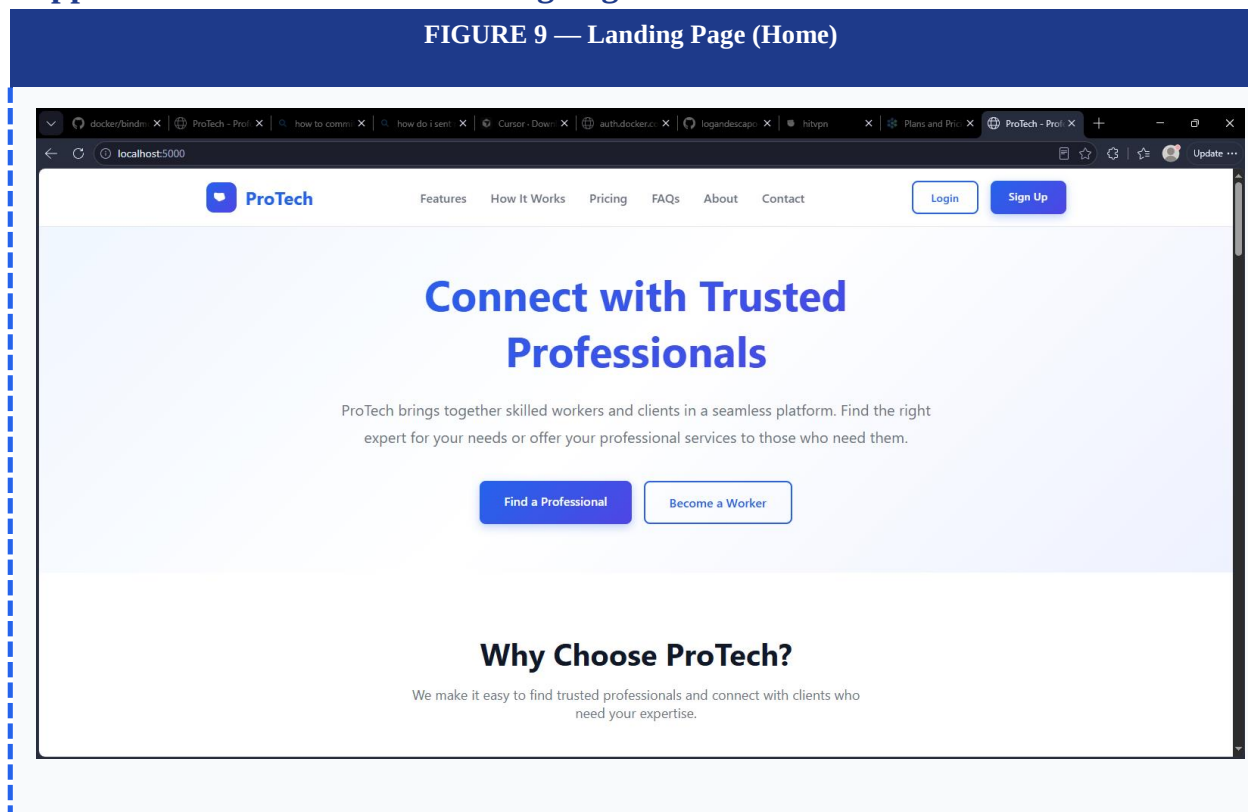
Appendix C — Application Screenshots (Insert Images)

This appendix leaves space for screenshots of every major ProTech screen. The dashed blue border marks where each image goes. To fill them in: start the app with docker compose up, open the URL shown in each figure, capture the screen (Win+Shift+S on Windows, Cmd+Shift+4 on Mac), click inside the placeholder in Word, and insert the image. Resize to fit the frame while keeping the aspect ratio. Good screenshots make the project defence much easier to follow.

Category	Screenshots to capture	Demo login
Public pages	Home, Auth, Pricing, FAQ	None required
Client journey	Dashboard, Browse, Book, Bookings, Wallet, Escrow, Review	john@example.com / password123
Worker journey	Dashboard, Bookings, Messages	mike@example.com / password123
Administration	Django Admin — Users, Bookings	Superuser account

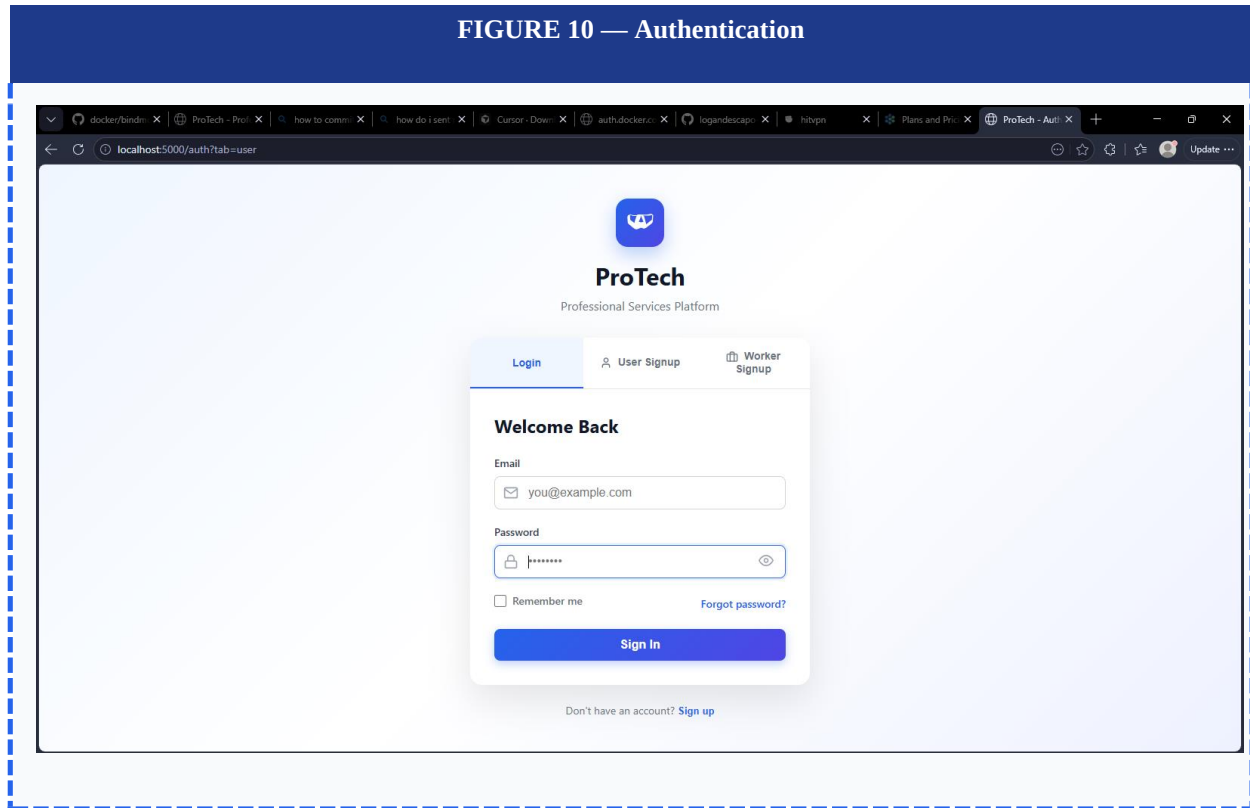
Appendix C — Public & Marketing Pages

FIGURE 9 — Landing Page (Home)



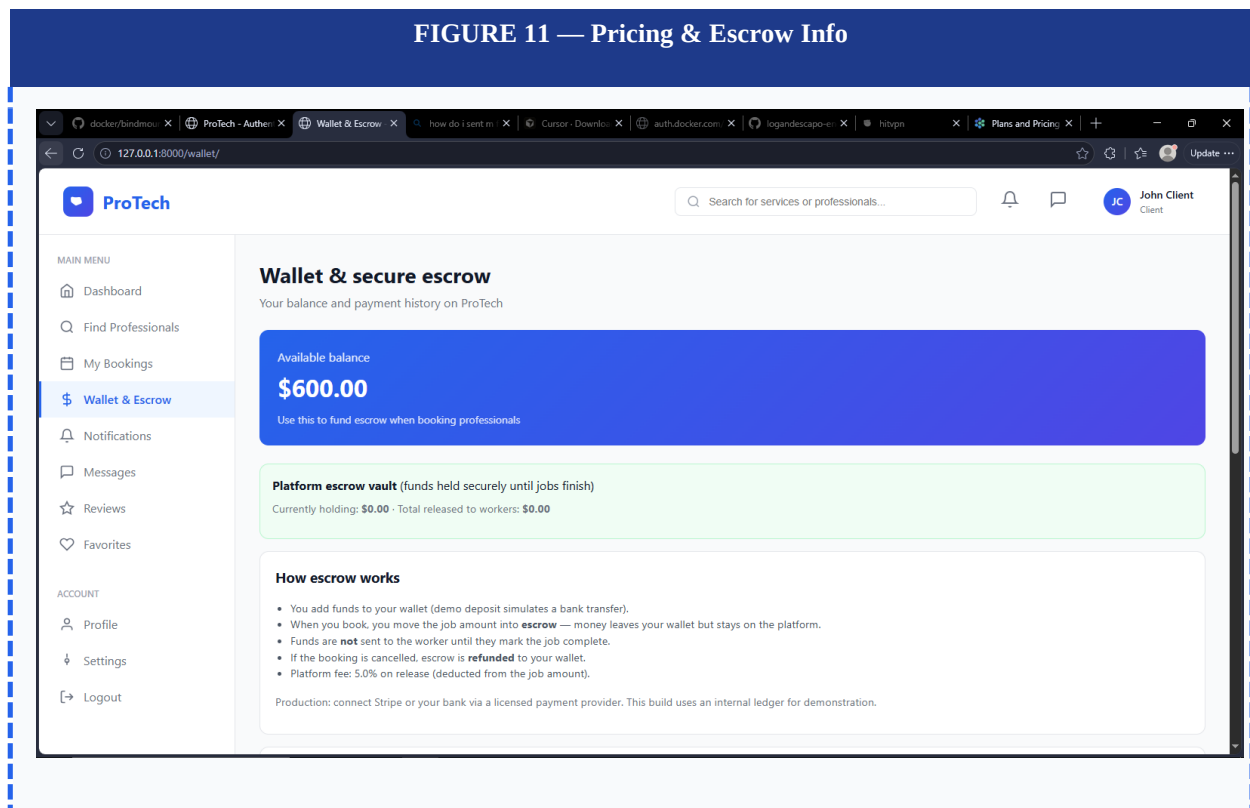
Full-width hero, feature highlights, and primary Sign Up / Find a Professional CTAs.

FIGURE 10 — Authentication



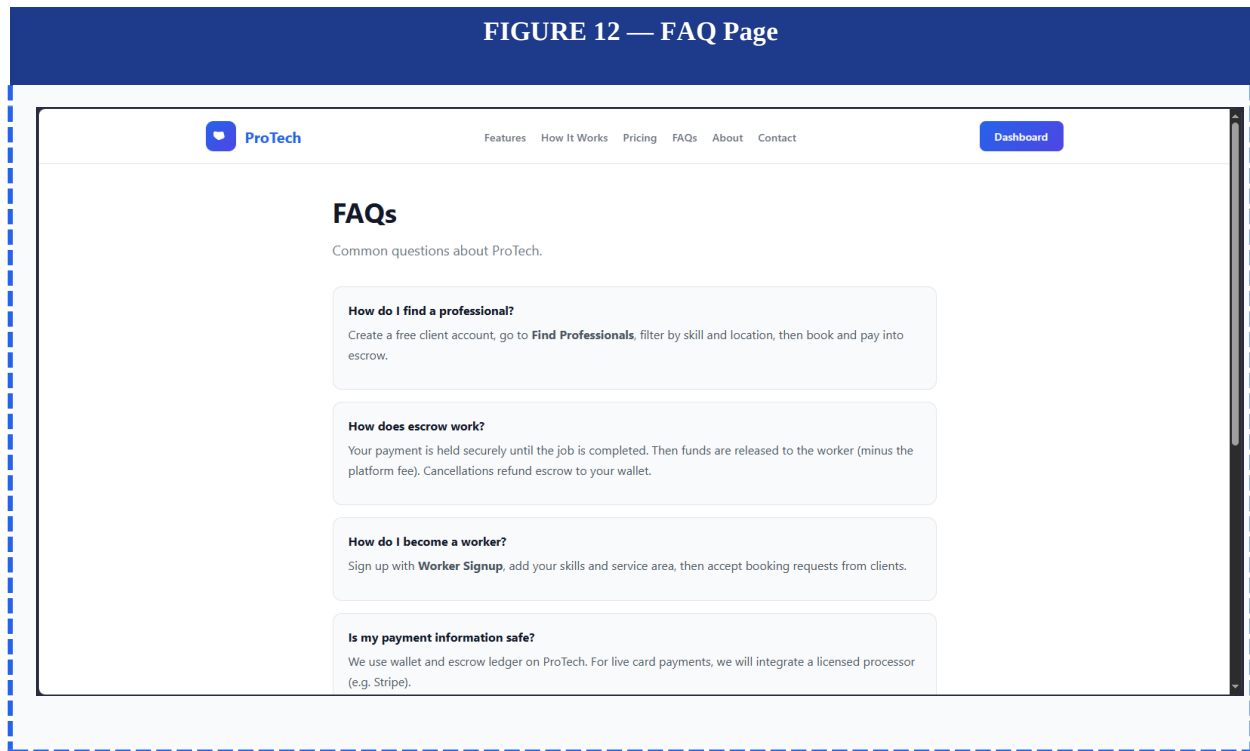
Login, client signup, and worker signup tabs with form validation.

FIGURE 11 — Pricing & Escrow Info



Landing sub-page explaining platform fees and escrow protection.

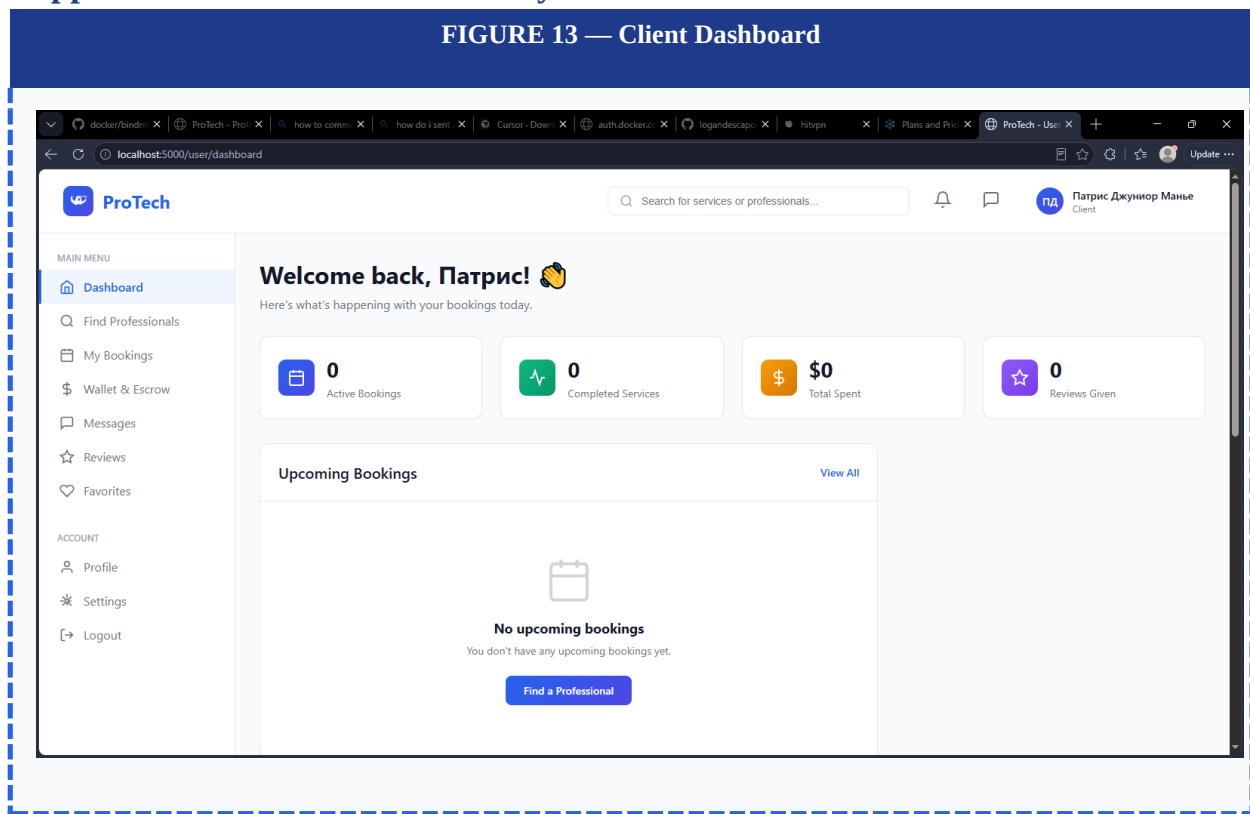
FIGURE 12 — FAQ Page



Frequently asked questions for clients and workers.

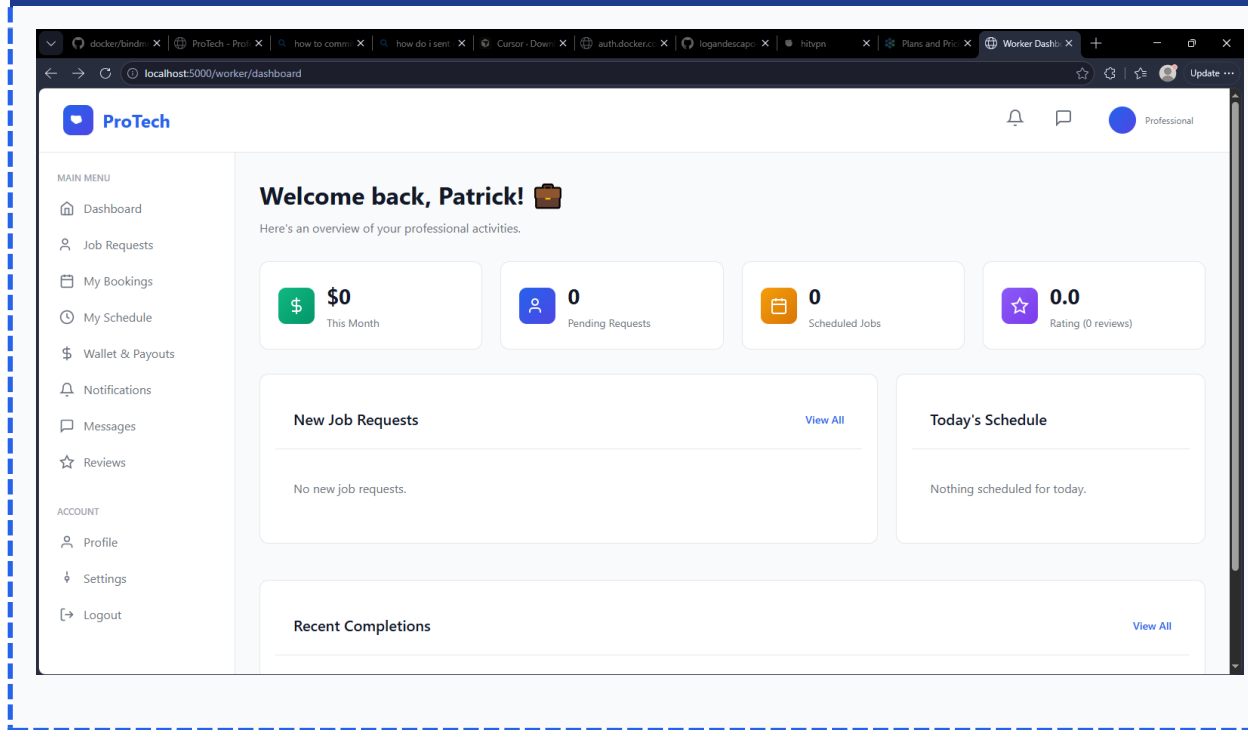
Appendix C — Client User Journey

FIGURE 13 — Client Dashboard



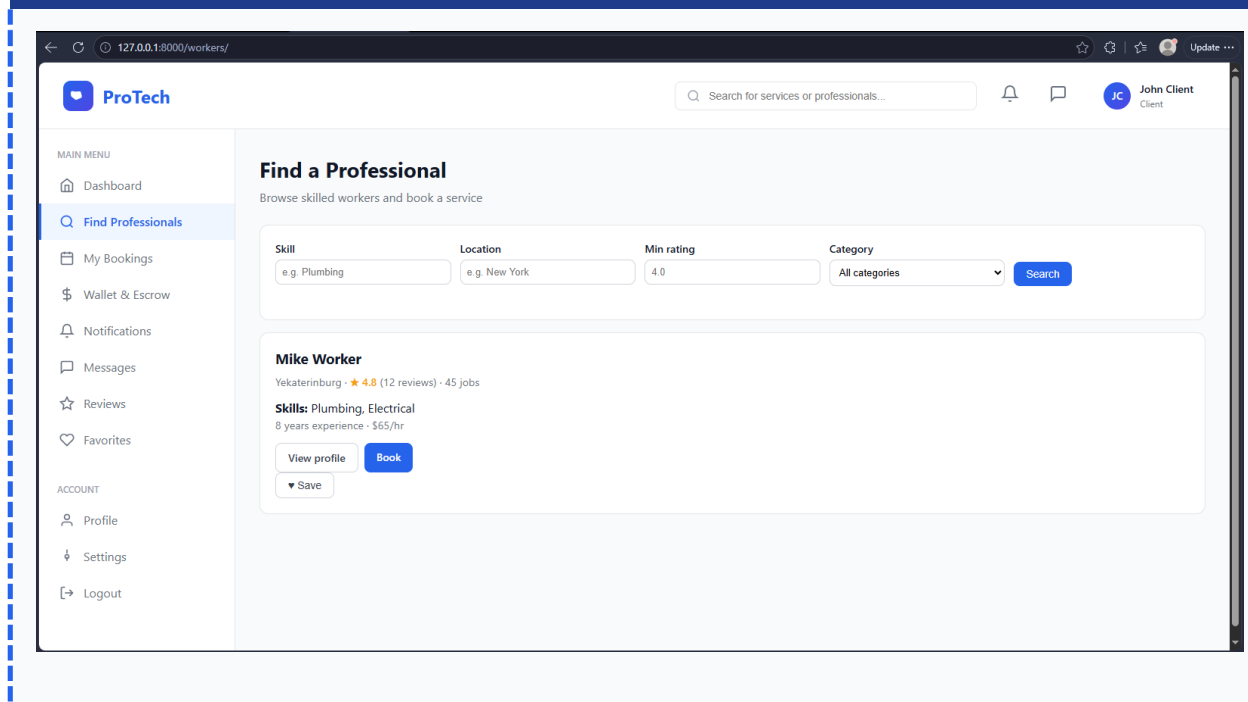
Upcoming bookings, quick actions, sidebar navigation, and notification badges.

FIGURE 14 — Browse Workers



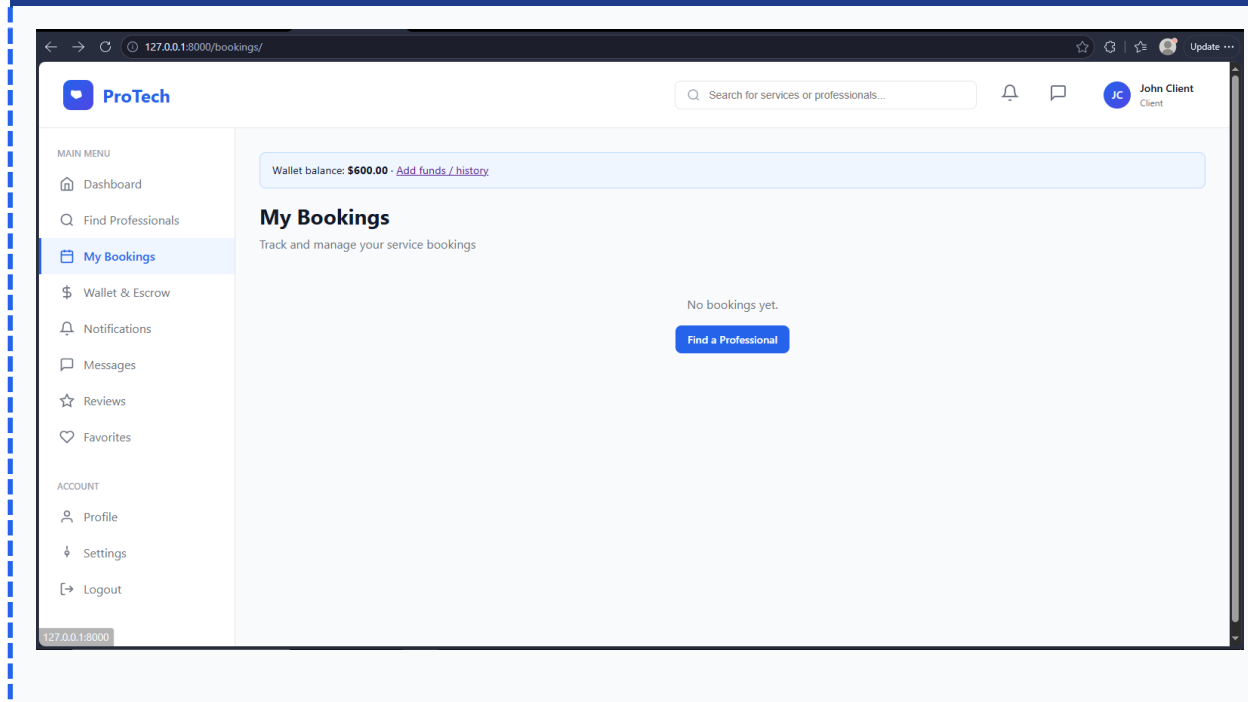
Search filters by skill, location, and rating; worker cards with Book action.

FIGURE 15 — Book a Worker



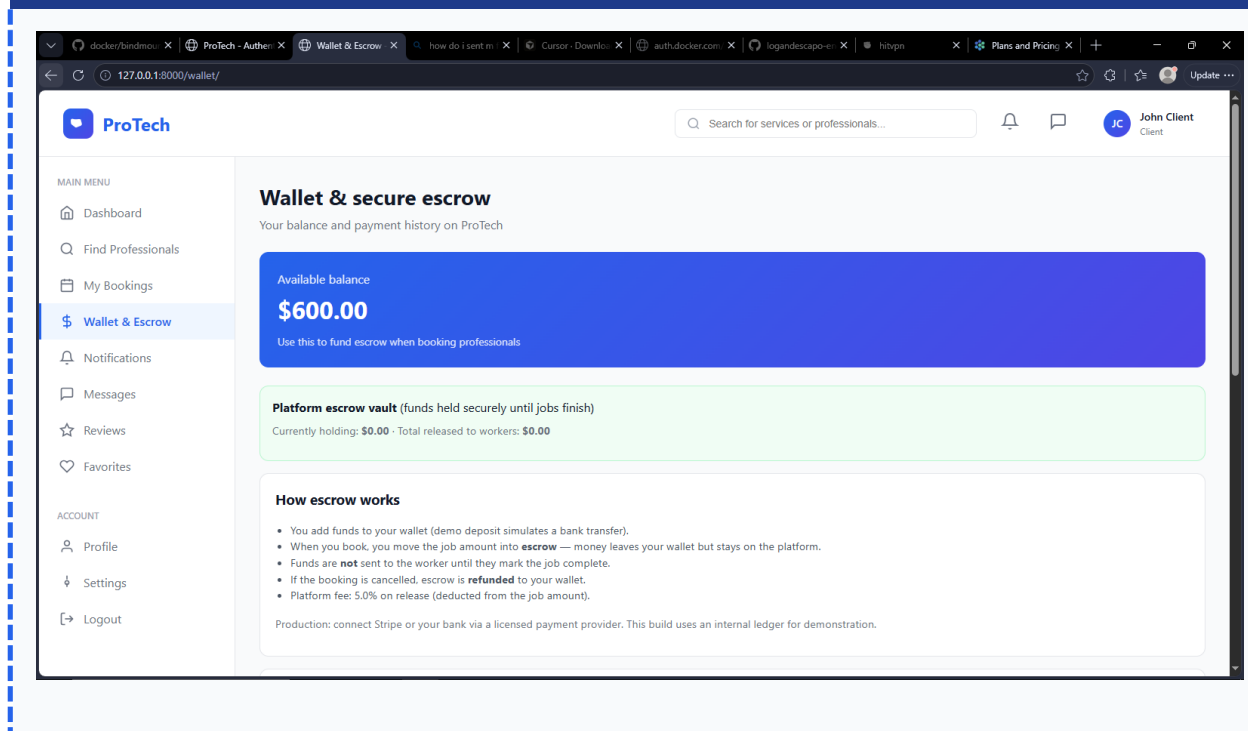
Booking form: date, time, address, description, and price fields.

FIGURE 16 — My Bookings



Booking list with status badges, escrow pay, complete, and review actions.

FIGURE 17 — Wallet



Demo deposit form, available balance, and transaction history.

FIGURE 18 — Escrow Payment

The screenshot displays the ProTech wallet interface in a browser window. The address bar shows '127.0.0.1:8000/wallet/'. The ProTech logo is in the top left. A search bar with the placeholder 'Search for services or professionals...' is in the top right. A user profile for 'John Client' is also visible. The main content area is divided into two sections: 'Add funds (demo)' and 'Transaction history'. The 'Add funds (demo)' section includes a sub-header 'Simulates a secure deposit from your bank. Max \$1,000 per transaction.', an 'Amount (\$)' input field with '100.00' entered, and a 'Deposit to wallet' button. The 'Transaction history' section lists three transactions, all labeled 'Deposit' and 'Demo bank deposit'.

Transaction history
Deposit Demo bank deposit Jun 09, 2026 08:28 AM + \$200.00 · Balance after: \$600.00
Deposit Demo bank deposit Jun 09, 2026 08:26 AM + \$200.00 · Balance after: \$400.00
Deposit Demo bank deposit

Pay-into-escrow confirmation showing amount and platform fee.

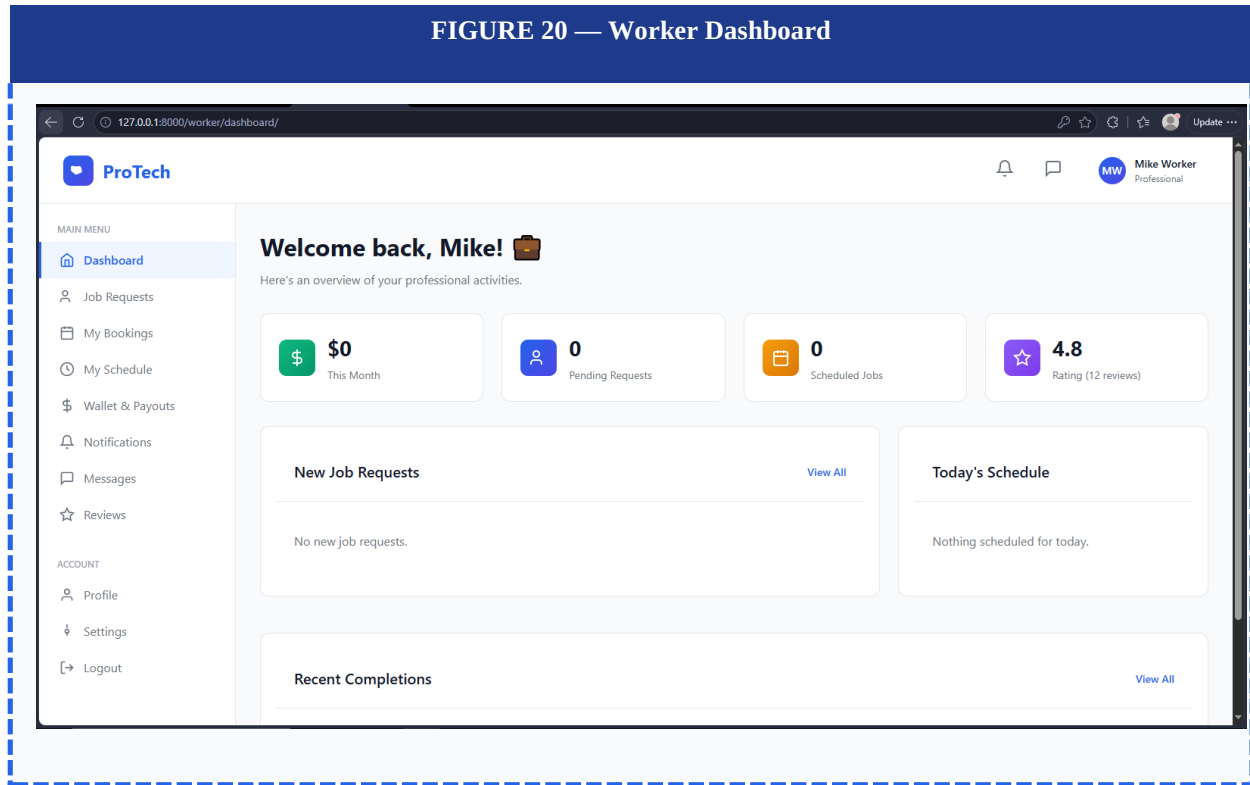
FIGURE 19 — Leave a Review

The screenshot shows the 'Leave a Review' form in the ProTech interface. The form is titled 'Leave a Review' and includes the sub-header 'Rate your experience'. It features a star rating system with five stars. The form is set against a light blue background.

Star rating and comment form after job completion.

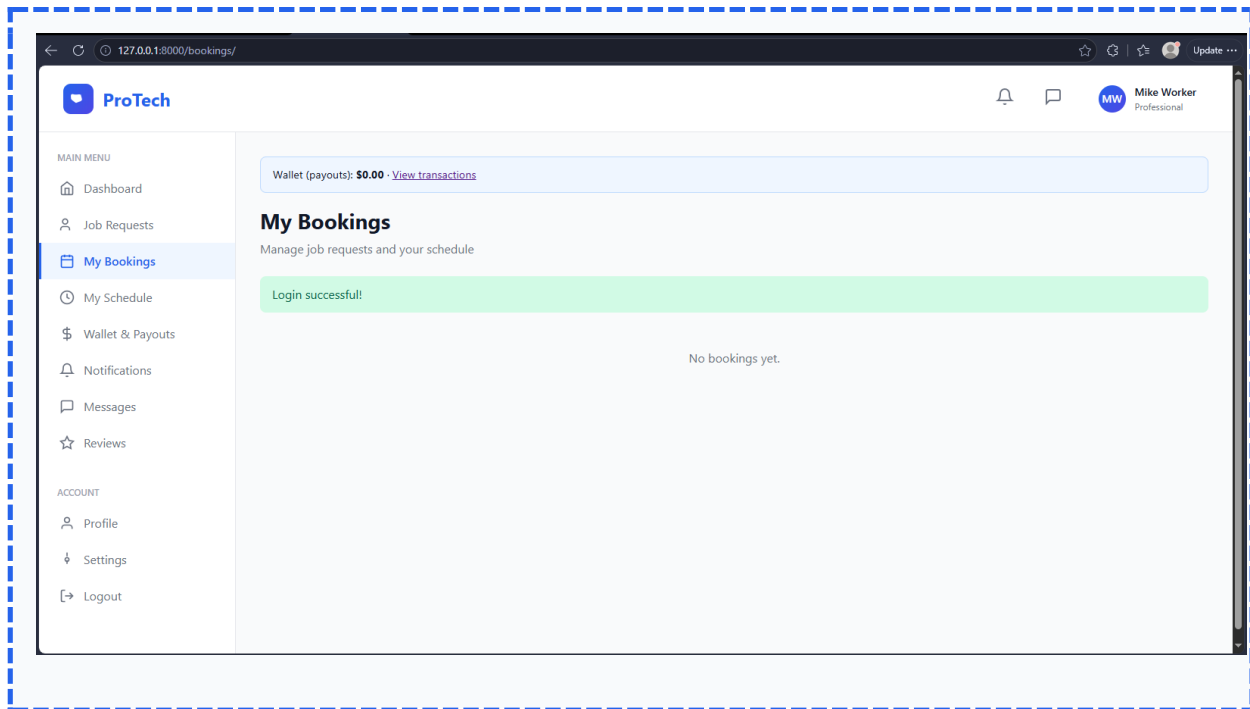
Appendix C — Worker User Journey

FIGURE 20 — Worker Dashboard



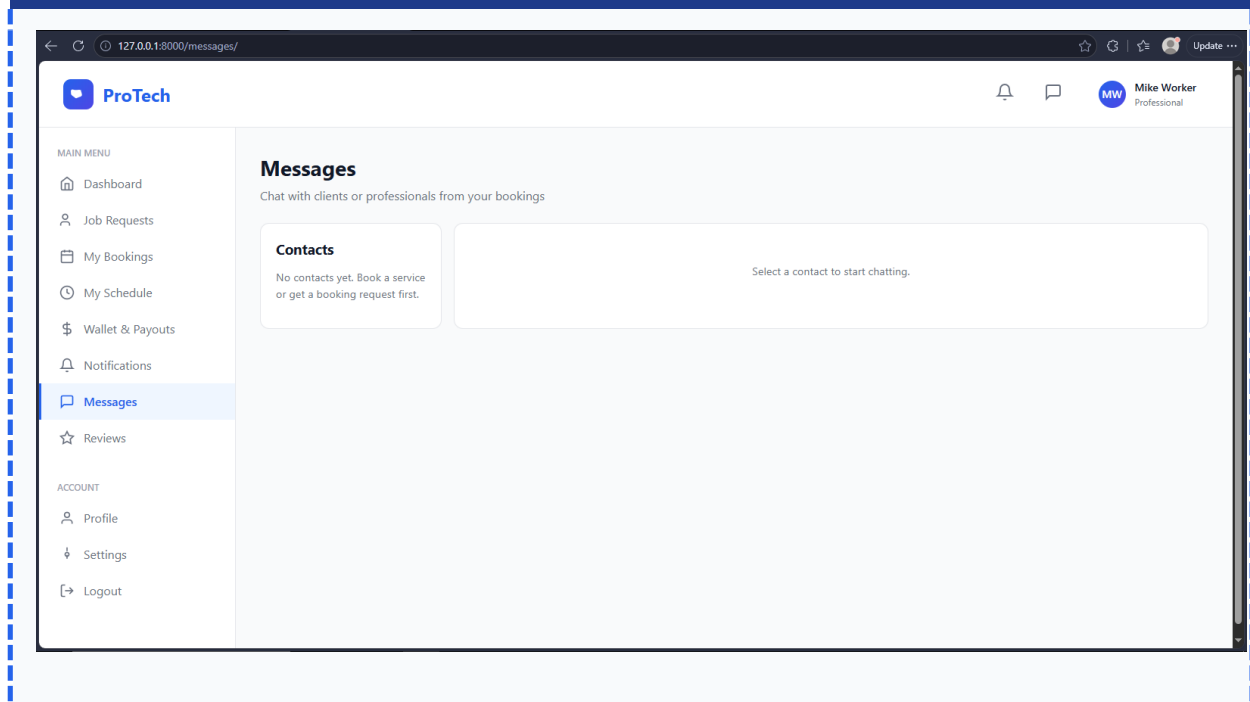
Pending requests, today's schedule, earnings summary, and job actions.

FIGURE 21 — Worker Bookings



Incoming jobs with Accept, Start, and Complete controls.

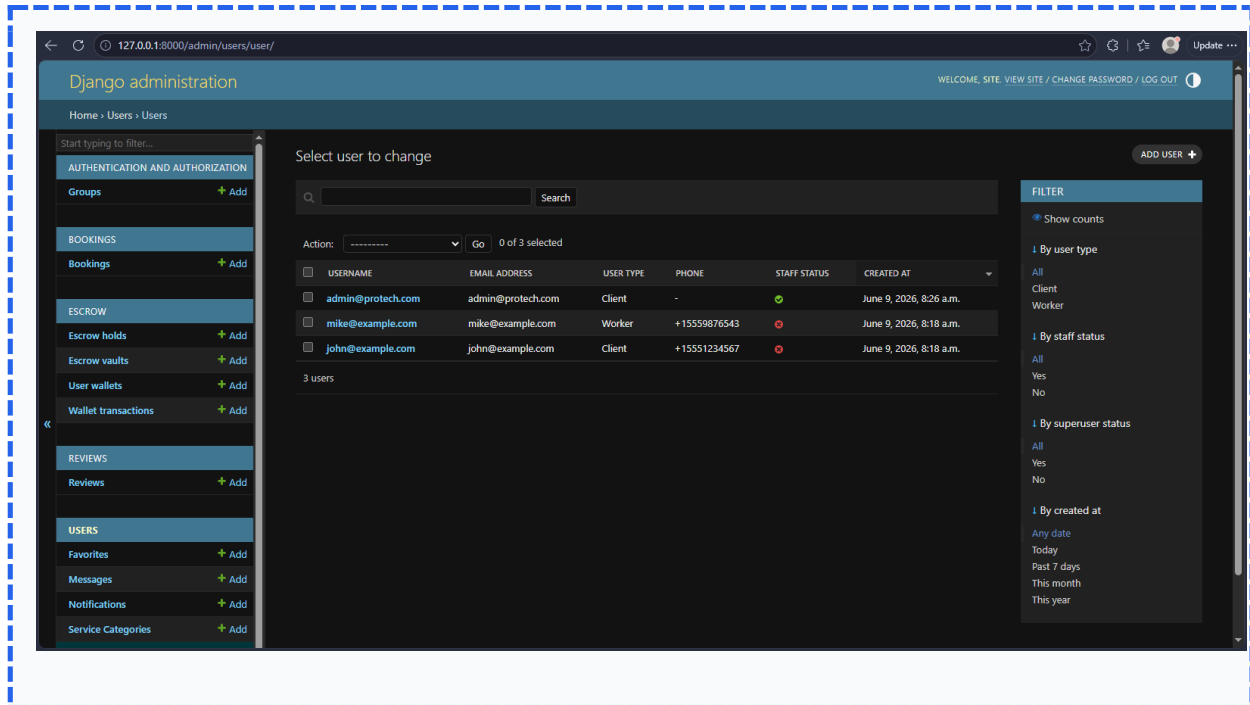
FIGURE 22 — Messages



In-app messaging between client and worker.

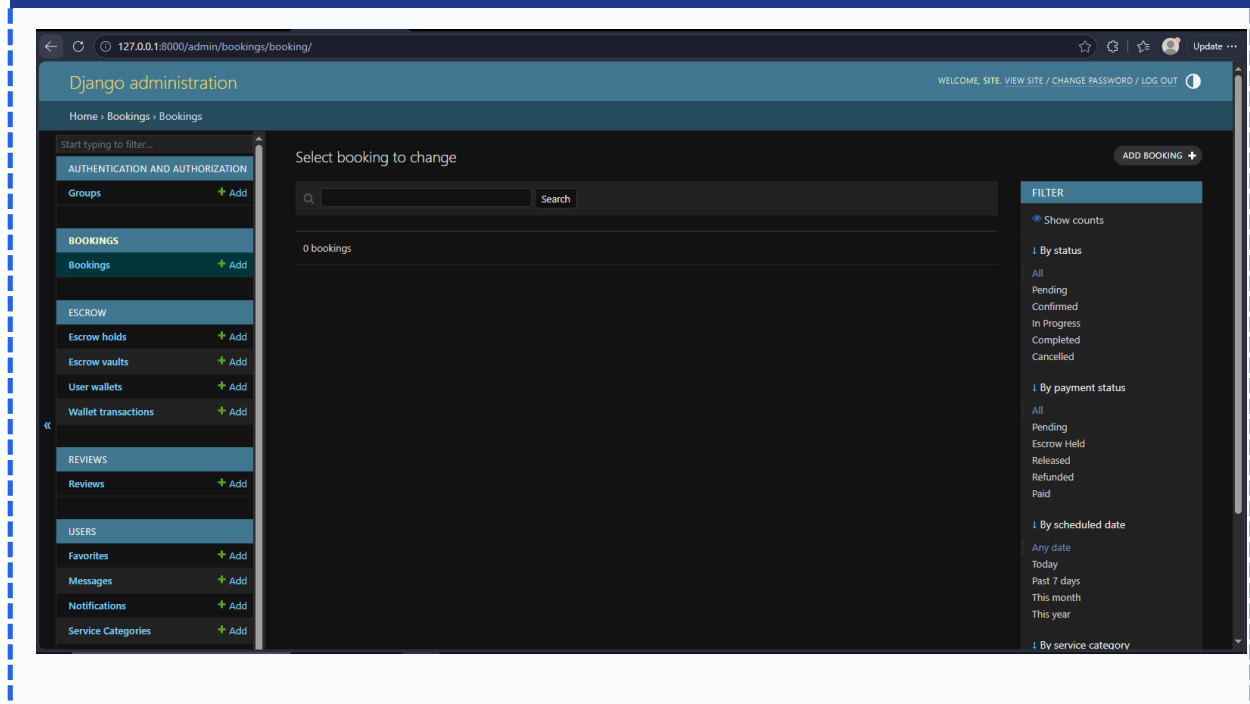
Appendix C — Administration (Django Admin)

FIGURE 23 — Django Admin — Users



Staff interface for managing accounts, roles, and permissions.

FIGURE 24 — Django Admin — Bookings



CRUD view of all platform bookings and payment statuses.

After inserting images, right-click → Wrap Text → In Line with Text or Square, then resize to stay inside the placeholder. Use Picture Format → Picture Border if you want a consistent look across figures.

Appendix D — Rubric Alignment (200 Points)

The table below maps each full-stack website development requirement from the project rubric to the concrete implementation in this repository. Assessors can verify each row via the cited path, the running application, or the automated test suite.

Requirement (rubric)	Implementation	Evidence
Django + templates	Django 5.2.15 with Jinja2 backend in templates/	Browse http://localhost ; see protech_project/jinja2_env.py
Auth & RBAC	Registration, login, logout, @user_type_required	users/decorators.py ; login as john@ / mike@
Password reset	/password-reset/ with Django token flow	users/urls.py ; console email in dev
Admin / CMS	Django Admin for all models	http://localhost/admin/
Security	CSRF, hashed passwords, .env secrets, ORM, XSS escaping	settings.py ; .gitignore; Section 3.3
Unit + integration tests	python manage.py test — 17 tests	users/tests_integration.py ; escrow/tests_integration.py
Docker multi-service	db, redis, backend, nginx in docker-compose.yml	<code>docker compose up --build</code>
Logging	Per-app loggers → stdout + logs/django.log	protech_project/settings.py LOGGING
Redis caching	Worker/category cache in cache_utils.py	protech_project/cache_utils.py
README & linting	README.md + flake8/black in CI	.github/workflows/ci.yml

To reproduce test evidence locally, run: `set USE_LOCMEM_CACHE=1 (Windows) or USE_LOCMEM_CACHE=1 python manage.py test`. CI output is available under the GitHub Actions tab of the project repository.